KUBERNETES ve KONTEYNER ORKESTRASYONU

İÇİNDEKİLER

KUBERNETES ve KONTEYNER ORKESTRASYONU 1
Tablolar Listesi 2
Şekiller Listesi 2
1. Dersin Amacı ve Öğrenme Çıktıları 3
1.1. Dersin amacı 3
1.2. Öğrenme Çıktıları 3
2. Kubernetes ve Küme Yönetimi 3
2.1. Orkestrasyon ve Bildirimsel Yönetim 4
2.2. Küme Mimarisi ve Temel Nesneler 5
3. Kubernetes ile Çalışmaya Başlama 8
3.1. Yerel ve Bulut Çalışma Ortamları 8
3.2. Araçlar ve Temel Yönetim Akışı 10
4. Uygulamanın Dağıtılması ve Erişime Açılması 11
4.1. Konteyner İmajından Çalışan Uygulamaya 11
4.2. Service ile İletişim ve Uygulamaya Erişim 12
4.3. Yapılandırma, Gizli Bilgiler ve Kalıcı Veri 14
5. Uygulamanın Sağlıklı Çalışması ve Ölçeklenmesi 16
5.1. Sağlık Kontrolleri ve Hata Davranışı 16
5.2. Kaynak Gereksinimleri ve Ölçekleme 18
5.3. Uygulamanın Güncellenmesi ve Önceki Sürüme Dönüş 20
6. Bulut Ortamında Kubernetes 22
6.1. Yerel Küme ve Yönetilen Kubernetes Hizmetleri 22
6.2. Uygulamanın Bulut Ortamına Uyarlanması 24
6.3. Bulutta Doğrulama ve Kaynak Yönetimi 26
7. Uygulama Çalışması 27
7.1. Yerel Uygulama 28
7.2. Bulut Uygulaması 28
7.3. Doğrulama, Teslim ve Değerlendirme 29
8. Bölüm Özeti 31
Kaynakça 32
Hafta 6 – Çalışma Soruları 35

Tablolar Listesi

Tablo 1. Kubernetes kümesinin temel bileşenleri ve görevleri 6
Tablo 2. Yerel ve bulut Kubernetes çalışma ortamları. 9
Tablo 3. Kubernetes uygulamasının temel yönetim süreci 10
Tablo 4. Web uygulamasının dağıtımında tanımlanan temel bilgiler. 12
Tablo 5. Temel Service türleri ve erişim amaçları. 14
Tablo 6. Uygulama bilgilerinin türleri ve Kubernetes karşılıkları. 16
Tablo 7. Kubernetes sağlık kontrollerinin amaçları ve başarısızlık sonuçları. 17
Tablo 8. Tek web konteyneri için örnek kaynak talepleri ve sınırları. 19
Tablo 9. Temel Deployment güncelleme stratejileri. 21
Tablo 10. Yerel Kubernetes ortamı ile buluttaki yönetilen Kubernetes hizmetlerinin temel farkları. 23
Tablo 11. Bir uygulamanın buluta uyarlanmasında korunan yapı ve değerlendirilen ayarlar. 25
Tablo 12. Bulutta çalışan bir uygulamanın temel doğrulama ölçütleri. 26

Şekiller Listesi

Şekil 1. Kubernetes kontrol düzlemi ve işçi düğümlerin ilişkisi. 7

1. Dersin Amacı ve Öğrenme Çıktıları

1.1. Dersin amacı

Bu haftanın amacı, konteyner hâline getirilmiş bir uygulamanın Kubernetes üzerinde nasıl dağıtıldığını ve yönetildiğini öğretmektir. Container orchestration (konteyner orkestrasyonu), uygulama kopyalarının çalıştırılması, erişime açılması, güncellenmesi ve kaynaklarının yönetilmesini birlikte ele alır. Kubernetes, uygulamanın kaç kopyasının çalışacağı ve hangi imajın kullanılacağı gibi bilgileri tanımlayan nesneleri kullanarak uygulamayı yönetir.

Önceki hafta ele alınan konteyner imajı oluşturma ve çalıştırma yaklaşımı, bu haftanın temelini oluşturur. Kubernetes üzerinde dağıtım, bileşenler arası iletişim, yapılandırma, kalıcı depolama ve sağlık kontrolleri birlikte değerlendirilir. Ayrıca uygulama kopyalarının ölçeklenmesi, sürüm güncellemeleri ve önceki sürüme dönüşün nasıl yönetildiği açıklanır.

Yerel Kubernetes ortamları ve yönetilen bulut hizmetleri, ortak kavramlar ve ortama bağlı farklılıklar açısından ele alınır. Öğrencinin, Kubernetes nesnelerinin görevlerini açıklayabilmesi ve bir uygulamanın farklı ortamlarda çalıştırılması için gereken uyarlamaları ayırt edebilmesi hedeflenir.

1.2. Öğrenme Çıktıları

Bu bölümün sonunda öğrenci:

1. Konteyner orkestrasyonunun amacını, Kubernetes kümesinin temel bileşenlerini ve bildirimsel yönetim yaklaşımını açıklar.
2. Yerel veya yönetilen bulut ortamında Kubernetes ile çalışmak için gerekli araçları ve temel yönetim akışını tanımlar.
3. Hazır bir konteyner imajını Pod ve Deployment nesneleriyle çalıştırır; Service üzerinden uygulamaya erişim sağlar.
4. Uygulamanın yapılandırmasını ve gerekli kalıcı veri bağlantılarını Kubernetes nesneleriyle düzenler.
5. Sağlık kontrollerini, kaynak taleplerini ve kopya sayısını belirler; temel hata ve güncelleme davranışlarını yorumlar.
6. Bir uygulamanın yerel Kubernetes ortamından buluta taşınmasında gereken uyarlamaları ve çalışma sonunda uygulama, küme ve depolama kaynaklarının yaşam döngülerini ayırt eder.

2. Kubernetes ve Küme Yönetimi

Bir uygulamayı konteyner hâline getirmek, onun farklı ortamlarda çalıştırılmasını kolaylaştırır. Ancak uygulamanın birden fazla kopyasını çalıştırmak, arızalara müdahale etmek ve sürümleri güncellemek, bu kopyaların birlikte yönetilmesini gerektirir.

Kubernetes, bu yönetimi uygulamanın istenen durumunu tanımlayan nesneler üzerinden gerçekleştirir. Bu bölümde orkestrasyon, bildirimsel yönetim ve Kubernetes kümesinin temel yapısı ele alınır.

2.1. Orkestrasyon ve Bildirimsel Yönetim

Bir konteyner imajının hazırlanması ve çalıştırılması, uygulamanın yönetim gereksinimlerinin tamamını karşılamaz. Örneğin bir web uygulamasının üç kopyasının çalışması, başarısız bir kopyanın yerine yenisinin oluşturulması ve yeni sürüme geçerken hizmetin sürdürülmesi gerekebilir. Bu işlemlerin farklı bilgisayarlarda elle yürütülmesi, uygulamanın durumunu takip etmeyi ve tutarlı biçimde yönetilmesini güçleştirir.

Orkestrasyon, bu işlemlerin ortak bir yönetim altında koordine edilmesidir. Kubernetes, konteyner uygulamalarının dağıtılmasını, ölçeklenmesini ve yönetilmesini sağlayan açık kaynaklı bir platformdur. Başlıca görevleri şunlardır:

- Uygulama kopyalarının uygun bilgisayarlarda çalıştırılmasını sağlamak.
- Tanımlanan kopya sayısının korunmasını sağlamak.
- Uygulamaya ağ üzerinden erişimi düzenlemek.
- Sürüm güncellemelerini ve yapılandırmayı yönetmek.
- Hata durumlarında gerekli yeniden başlatma veya yeniden oluşturma işlemlerini gerçekleştirmek.

Kubernetes, uygulamanın kaynak kodunu yazmaz veya konteyner imajını oluşturmaz. Önceden hazırlanmış imajları kullanarak uygulama kopyalarının çalıştırılmasını ve yönetilmesini sağlar. Bu nedenle imajın hazırlanması ile uygulamanın orkestrasyonu, birbirini tamamlayan farklı süreçlerdir. [1]

Kubernetes'in bu görevleri yerine getirebilmesi için uygulamanın nasıl çalışması gerektiğinin tanımlanması gerekir. Örneğin hangi imajın kullanılacağı ve kaç uygulama kopyasının çalışacağı belirtilir. Kubernetes, çalışan uygulamanın durumunu bu tanımla karşılaştırarak gerekli düzenlemeleri yapar. Bu çalışma biçiminin temelinde bildirimsel yönetim yaklaşımı bulunur.

Bildirimsel yönetim yaklaşımı

Declarative management (bildirimsel yönetim), sistemin nasıl görünmesi gerektiğini tanımlamaya dayanır. Kullanıcı, uygulamanın hangi imajdan çalışacağını ve kaç kopyasının bulunacağını belirtir. Bu tanım, "desired state" (istenen durum) olarak adlandırılır. Kubernetes, bu bilgileri nesneler aracılığıyla saklar. Nesne burada uygulamayı veya onun bir gereksinimini temsil eden yapılandırılmış bir kayıttır. [2]

Örneğin bir web uygulaması için şu hedef belirlenebilir:

Uygulamanın belirli bir imaj sürümünden üç kopya çalışsın.

Bu ifade, kopyaların hangi sırayla başlatılacağını veya bir hata sonrasında hangi komutların çalıştırılacağını tek tek belirtmez. Hedefi tanımlar; bu hedefin gerçekleştirilmesi için gereken işlemleri Kubernetes'in yönetim bileşenleri yürütür.

Imperative management (buyruksal yönetim) ise yapılacak işlemi doğrudan belirtir. "Yeni bir kopya oluştur" bir işlem talebidir; "toplam üç kopya bulunsun" ise bir durum tanımıdır. Kubernetes her iki yönetim biçimini de destekler. Bildirimsel yaklaşımda istenen durumun dosyalarda saklanması, tanımların incelenmesini, tekrar kullanılmasını ve değişikliklerin izlenmesini kolaylaştırır.

İstenen durumun korunması

Kubernetes'in controller (denetleyici) adı verilen bileşenleri, istenen durum ile gözlenen durumu karşılaştırır. Aradaki farkı azaltmak için gerekli işlemleri başlatır. Bu sürekli karşılaştırma ve düzeltme sürecine reconciliation (durum uzlaştırma) denir. [3]

Üç kopyalı uygulama örneğinde süreç şöyle işler:

1. Kullanıcı, hedef kopya sayısını üç olarak tanımlar.
2. Kubernetes, mevcut kopyaların durumunu izler.
3. Yönetilen kopyalardan biri silinirse gözlenen sayı ikiye düşer.
4. İlgili denetleyici, eksik kopyanın yerine yenisinin oluşturulmasını sağlar.
5. Yeni kopya çalıştığında gözlenen durum yeniden hedefe yaklaşır.

Bu yaklaşımın önemli sonucu, hedefin yalnız ilk dağıtım sırasında kullanılmamasıdır. Yönetim nesnesi varlığını sürdürdükçe Kubernetes hedefi korumaya çalışır. Hedef iki kopyaya değiştirilirse fazla kopyaların azaltılması da aynı mantıkla yönetilir.

Bununla birlikte, istenen durumun tanımlanması onun hemen gerçekleşeceğini garanti etmez. Yeterli kaynak bulunmaması veya imaja erişilememesi, yeni kopyanın çalışmasını engelleyebilir. Ayrıca bir sürecin çalışması, uygulamanın kullanıcı isteklerini doğru yanıtladığını tek başına göstermez. Bu nedenle yönetimde, tanımlanan hedef ile gerçekleşen durum ve uygulamanın hizmet verebilmesi birlikte değerlendirilir.

2.2. Küme Mimarisi ve Temel Nesneler

Kubernetes, uygulamaları *cluster (küme)* adı verilen bir yapı üzerinde yönetir. Küme, uygulamaları çalıştıran bilgisayarlar ile bunların yönetimini sağlayan bileşenlerden oluşur. Kümedeki fiziksel veya sanal bir bilgisayar *node (düğüm)* olarak adlandırılır.

Mimaride iki temel rol bulunur. *Control plane (kontrol düzlemi)*, kümenin yönetiminden ve istenen durumun korunmasına yönelik kararlardan sorumludur. *Worker node (işçi düğüm)*

ise uygulama konteynerlerini çalıştırır. Yerel eğitim ortamında bu roller aynı bilgisayarda bulunabilir; birden fazla bilgisayardan oluşan kümelerde farklı düğümlere dağıtılabilir. [4]

Kontrol düzlemi ve düğüm bileşenleri

Kullanıcı, uygulama tanımlarını Kubernetes'in *API — Application Programming Interface (uygulama programlama arayüzü)* üzerinden kümeye iletir. Kontrol düzlemi bu tanımları saklar ve gerekli yönetim işlemlerini yürütür. Düğüm tarafındaki bileşenler ise kendilerine atanan uygulamaların çalışmasını sağlar. Temel görev dağılımı Tablo 1'de özetlenmiştir.

Tablo 1. Kubernetes kümesinin temel bileşenleri ve görevleri

Konum	Bileşen	Temel görev
Kontrol düzlemi	kube-apiserver	Nesne oluşturma, güncelleme ve sorgulama işlemleri için API erişimini sunar.
Kontrol düzlemi	etcd	Kümenin nesnelerine ait durum bilgilerini saklar.
Kontrol düzlemi	kube-scheduler	Henüz yerleştirilmemiş Pod'lar için uygun düğüm seçer.
Kontrol düzlemi	kube-controller-manager	İstenen durumun korunmasını sağlayan denetleyicileri çalıştırır.
İşçi düğüm	kubelet	Düğüme atanan Pod'ların tanımlarına uygun çalışmasını sağlar.
İşçi düğüm	Konteyner çalışma zamanı	Konteynerleri başlatır ve yürütür.

Container runtime (konteyner çalışma zamanı) için containerd bir örnektir. Docker ile hazırlanmış bir imajın Kubernetes üzerinde kullanılması, her düğümde Docker Desktop bulunmasını gerektirmez. Ağ iletişimi için de ek bileşenler kullanılır; bunlar ağ bölümünde ele alınacaktır. [4]

Şekil 1, yönetim bileşenleri ile uygulamaların çalıştığı düğümler arasındaki ilişkiyi gösterir. Çizgiler yönetim iletişimini temsil eder; kullanıcıların uygulamaya gönderdiği isteklerin yolu bu şekilde gösterilmemiştir.

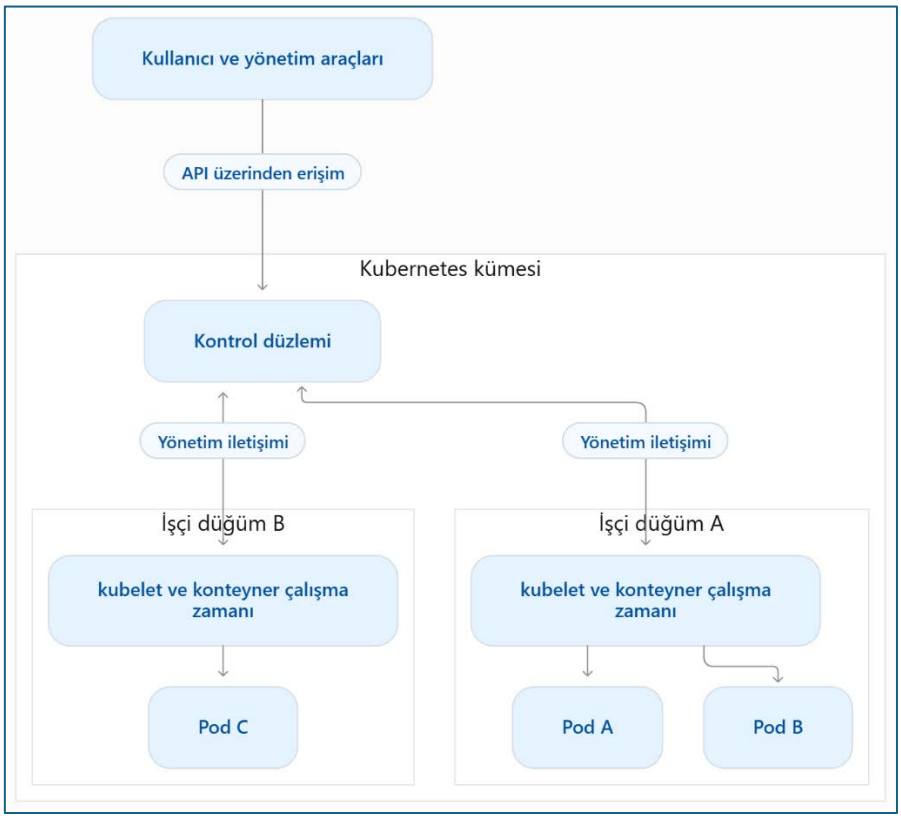
Pod ve konteyner ilişkisi

Kubernetes'in uygulama çalıştırmak için oluşturduğu en küçük birim *Pod*'dur. Bir Pod, aynı düğüm üzerinde birlikte çalışan bir veya daha fazla konteyneri barındırır. Başlangıç düzeyindeki uygulamalarda çoğunlukla bir Pod içinde tek uygulama konteyneri bulunur. [5]

Bir Pod içindeki konteynerler aynı ağ bağlamını paylaşır ve localhost üzerinden iletişim kurabilir. Tanımlanmış ortak depolama birimlerini de kullanabilirler. Bu nedenle aynı Pod'a yerleştirilecek konteynerlerin birlikte çalışması ve aynı yaşam döngüsünü paylaşması beklenir.

Bir web uygulamasının üç bağımsız kopyasını çalıştırmak istediğimizi düşünelim. Her biri web konteynerini barındıran üç ayrı Pod oluşturulur. Üç bağımsız uygulama kopyasını tek Pod içine yerleştirmek aynı yönetim ve ölçekleme davranışını sağlamaz; aynı Pod içindeki konteynerler birlikte yerleştirilir ve Pod düzeyinde birlikte yönetilir.

Pod kalıcı bir bilgisayar gibi düşünülmemelidir. Silinen bir Pod'un yerine oluşturulan Pod yeni bir nesnedir. Bu nedenle uygulamanın kalıcı verisi, yalnız Pod'un geçici yaşamına bağlı tutulmamalıdır. [5]



Şekil 1. Kubernetes kontrol düzlemi ve işçi düğümlerin ilişkisi.

Deployment ve ReplicaSet ilişkisi

Pod'ları tek tek oluşturmak, uygulama kopyalarının korunması ve güncellenmesi için yeterli bir yönetim yaklaşımı değildir. Bu amaçla *Deployment (dağıtım nesnesi)* kullanılır. Deployment; kullanılacak imajı, Pod'ların nasıl oluşturulacağını ve hedef kopya sayısını tanımlar. Pod oluşturmak için kullanılan bu tanıma *Pod template (Pod şablonu)* denir. İmaj sürümü değiştirildiğinde Deployment, uygulamanın güncellenmesini de yönetir. [6]

Deployment, kopya sayısını korumak için *ReplicaSet (kopya kümesi)* nesnelerini yönetir. ReplicaSet'in görevi, belirlenen sayıda Pod'un bulunmasını sağlamaktır. Gerekli sayının altında Pod varsa yeni Pod oluşturulmasını sağlar; fazla Pod varsa sayının azaltılmasını sağlar. Uygulamada ReplicaSet'i ayrıca elle oluşturmak yerine Deployment üzerinden yönetmek genellikle yeterlidir. [7]

Bir web uygulaması için üç kopyalı Deployment oluşturulduğunda:

1. Deployment, uygulamanın Pod şablonunu ve kopya hedefini tanımlar.
2. Deployment'ın yönettiği ReplicaSet, üç Pod'un bulunmasını sağlar.
3. Pod'lar uygun düğümlere atanır ve konteynerleri çalıştırılır.
4. Pod'lardan biri silinirse ReplicaSet eksik kopyayı tamamlar.

Burada *imaj, Pod* ve *Deployment* farklı görevler üstlenir. İmaj, uygulamanın paketlenmiş hâlidir. Pod, bu imajdan oluşturulan konteynerin çalışma ortamıdır. Deployment ise Pod kopyalarının ve güncellemelerinin yönetimini sağlar.

Üç Pod bulunması, bunların mutlaka üç farklı düğüme yerleştirildiği anlamına gelmez. Örneğin tek düğümlü yerel ortamda bütün kopyalar aynı bilgisayarın kaynaklarını kullanır.

Ad alanları, etiketler ve seçiciler

Kümede farklı uygulamaların nesnelerini düzenlemek için namespace (ad alanı) kullanılır. Örneğin bir web uygulamasına ait nesneler, `uyg` adlı bir ad alanında tutulabilir. Ad alanları isimlerin ve nesnelerin düzenlenmesini kolaylaştırır; bütün kaynak türleri ad alanına bağlı değildir. Ayrıca ad alanı oluşturmak tek başına ağ veya güvenlik izolasyonu sağlamaz. [8]

Label (etiket), bir nesneye eklenen anahtar–değer bilgisidir. Örneğin web Pod'larına `app=web` etiketi verilebilir. *Selector (seçici)* ise etiketlere göre bir nesne grubunu belirler. Böylece yönetim nesneleri, ilgili Pod'ları adlarını tek tek bilmeden seçebilir. [9]

Deployment'ın seçicisi ile Pod şablonundaki etiketlerin uyumlu olması gerekir. Service nesnesi de erişim sağlayacağı Pod'ları seçmek için etiketlerden yararlanır. Örneğin seçicisi `app=web` olan bir Service, bu etikete sahip Pod'ları hedefler. Bu nedenle etiketler yalnız açıklayıcı bilgiler değil, nesneler arasındaki ilişkilerin kurulmasında kullanılan bilgilerdir.

3. Kubernetes ile Çalışmaya Başlama

Kubernetes üzerinde bir uygulama çalıştırabilmek için bir küme ve bu kümeye erişim sağlayan yönetim araçları gerekir. Küme, yerel bilgisayarda veya bir bulut sağlayıcısının altyapısında bulunabilir. Her iki ortamda da aynı temel Kubernetes nesneleri kullanılır; ortamın hazırlanması, bağlantı ve bazı altyapı gereksinimleri farklılaşır.

Bu bölümde yerel ve bulut çalışma ortamları, yönetim araçlarının görevleri ve temel nesne yönetim süreci açıklanır. Yerel ortamın kurulum ve hazırlık yönergeleri Ek 1'de yer alır.

3.1. Yerel ve Bulut Çalışma Ortamları

Yerel çalışma ortamı

Yerel Kubernetes ortamı, uygulamaların kişisel bilgisayarda geliştirilmesine ve denenmesine olanak verir. Bulut hesabı gerektirmeden dağıtım, iletişim, depolama ve

yönetim davranışları incelenebilir. Bu ortamlar özellikle öğrenme, geliştirme ve küçük ölçekli denemeler için uygundur.

Windows üzerinde Docker Desktop ve Minikube iki çalışma seçeneğidir. Docker Desktop, konteyner araçlarının yanında yerel Kubernetes kümesi oluşturma olanağı sunar. Ancak Docker Desktop'ın çalışıyor olması, Kubernetes kümesinin de hazır olduğu anlamına gelmez. Kubernetes ortamının ayrıca hazırlanması gerekir. [10]

Minikube, öğrenme ve geliştirme amacıyla yerel küme oluşturmayı kolaylaştıran bir araçtır. Kümenin çalışacağı altyapıyı seçmek için sürücüler kullanır. Kullanılacak sürücü, bilgisayardaki konteyner veya sanal makine ortamıyla uyumlu olmalıdır. [11]

Yerel küme, bilgisayarın işlemci, bellek ve disk kaynaklarını uygulamayla birlikte kullanır. Aynı bilgisayarda birden fazla düğüm oluşturulması da mümkündür; ancak bu düğümler aynı fiziksel makineye bağımlıdır. Dolayısıyla yerel deney, nesne yönetimini öğretirken farklı fiziksel makinelerde sağlanan hata toleransını temsil etmez.

Yönetilen bulut ortamı

Managed Kubernetes (yönetilen Kubernetes) hizmetlerinde sağlayıcı, kümenin işletilmesine ilişkin bazı görevleri üstlenir. Kullanıcı ise uygulama imajını, nesne tanımlarını, kaynak gereksinimlerini ve uygulama ayarlarını belirlemeye devam eder. Düğüm yönetiminin ne kadarının sağlayıcıya bırakıldığı, seçilen hizmet biçimine bağlıdır.

Yerel ortam araçları ve üç bulut sağlayıcısının yönetilen Kubernetes hizmetleri Tablo 2'de özetlenmiştir.

Tablo 2. Yerel ve bulut Kubernetes çalışma ortamları.

Ortam	Araç veya hizmet	Temel kullanım
Yerel bilgisayar	Docker Desktop Kubernetes	Docker Desktop üzerinden yerel küme oluşturma ve uygulama deneme
Yerel bilgisayar	Minikube	Öğrenme ve geliştirme amacıyla yerel küme oluşturma
AWS	Amazon Elastic Kubernetes Service — EKS	AWS altyapısında yönetilen Kubernetes kullanma
Microsoft Azure	Azure Kubernetes Service — AKS	Azure altyapısında yönetilen Kubernetes kullanma
Google Cloud	Google Kubernetes Engine — GKE	Google Cloud altyapısında yönetilen Kubernetes kullanma

Yerel ve bulut kümelerinde aynı temel nesne yönetim yaklaşımı kullanılır. Ancak imaj erişimi, dış ağ bağlantısı, depolama ve erişim yetkileri hedef ortama göre düzenlenir. Bulut kümesine bağlantı sağlayıcının araçlarıyla hazırlanabilir; uygulama nesneleri ise ortak Kubernetes araçlarıyla yönetilir. [12]–[14]

3.2. Araçlar ve Temel Yönetim Akışı

Yönetim aracı ve küme bağlantısı

Kubernetes'i komut satırından yönetmek için *kubectl* kullanılır. Bu araç, API üzerinden kümeye istek göndererek nesnelerin oluşturulmasını, sorgulanmasını, güncellenmesini ve silinmesini sağlar. Yerel bilgisayardan uzak bir bulut kümesi de yönetilebilir; yönetim aracının küme düğümünde çalışması gerekmez. [15]

Bağlantı bilgileri *kubeconfig* adı verilen yapılandırmada tutulur. Bu yapılandırma, küme adresi ve kullanıcı erişimi gibi bilgileri içerir. *Context (bağlam)*, kullanılacak küme, kullanıcı ve varsa varsayılan ad alanını birlikte seçer. Aynı bilgisayardan farklı kümeler yönetildiğinde etkin bağlam kontrol edilmelidir. Aksi hâlde bir ortam için amaçlanan oluşturma, güncelleme veya silme işlemi başka bir kümeye yöneltilebilir. [16]

Nesne tanımları ve yönetim süreci

Bildirimsel yönetimde uygulamanın gereksinimleri tanım dosyalarıyla ifade edilir. Bu dosyalarda yaygın kullanılan *YAML*, insan tarafından okunabilir bir veri gösterim biçimidir. Dosyalar; nesne türünü, kimlik bilgilerini ve istenen durumu belirtir. Böylece uygulama tanımı saklanabilir, incelenebilir ve yeniden kullanılabilir. [2]

Dosyanın kümeye uygulanması ile uygulamanın çalışmaya hazır olması farklı aşamalardır. Tanım kabul edilmiş olsa da imaj erişimi, kaynak yetersizliği veya uygulama hatası nedeniyle hedef durum gerçekleşmeyebilir. Bu nedenle yönetim süreci, oluşturma kadar gözlem ve doğrulamayı da içerir.

Durum incelemesinde nesne bilgileri, olay kayıtları ve konteyner günlükleri birlikte kullanılır. Nesne bilgileri mevcut durumu, olay kayıtları yerleştirme veya imaj indirme gibi işlemlerde karşılaşılan sorunları, konteyner günlükleri ise uygulamanın ürettiği mesajları gösterir. Bu bilgiler, uygulamanın neden başlamadığını veya hedef kopya sayısına neden ulaşamadığını anlamaya yardımcı olur. [17]

Tablo 3. Kubernetes uygulamasının temel yönetim süreci

Aşama	Amaç
Bağlantıyı doğrulama	Doğru kümeye erişildiğini ve gerekli izinlerin bulunduğunu belirlemek
İstenen durumu tanımlama	İmajı, kopya sayısını ve uygulama gereksinimlerini belirtmek
Tanımı uygulama	Nesnelerin oluşturulmasını veya güncellenmesini sağlamak
Durumu inceleme	İstenen durum ile gerçekleşen durumu karşılaştırmak
Değişikliği doğrulama	Güncelleme veya ölçekleme sonrasında uygulamanın davranışını değerlendirmek
Kaynakları kaldırma	Deneme tamamlandığında ilgili nesneleri ve altyapı kaynaklarını temizlemek

Uygulama nesnelerini kaldırmak, kümenin kendisini kaldırmak anlamına gelmez. Benzer şekilde Deployment tarafından yönetilen bir Pod'un silinmesi, onun yerine yeni bir Pod oluşturulmasına yol açabilir. Uygulamayı sonlandırırken onu yöneten nesneler de değerlendirilir; çalışma ortamını kaldırırken ise küme ve ilişkili altyapı kaynakları ayrıca ele alınır. Bu nedenle nesne, uygulama ve altyapı yaşam döngüleri birbirinden ayrılmalıdır.

4. Uygulamanın Dağıtılması ve Erişime Açılması

Hazırlanmış bir konteyner imajını Kubernetes üzerinde çalıştırmak için kullanılacak imaj, uygulama kopyalarının sayısı ve çalışma ayarları tanımlanır. Uygulamanın kullanıcılar tarafından erişilebilir olması, diğer bileşenlerle iletişim kurması ve gerekli verileri saklaması da ayrıca düzenlenir.

Bu bölümde imajın Pod ve Deployment üzerinden çalıştırılması, Service ile erişimin sağlanması, yapılandırma ve gizli bilgilerin uygulamaya verilmesi ve kalıcı depolamanın kullanılması ele alınır. Web uygulaması ve veritabanından oluşan bir sistem üzerinden bu gereksinimlerin birbirini nasıl tamamladığı açıklanır.

4.1. Konteyner İmajından Çalışan Uygulamaya

Konteyner imajı, uygulamanın kodunu ve çalışması için gerekli bağımlılıkları içerir. Ancak imajın hazır olması, uygulamanın Kubernetes üzerinde çalıştığı anlamına gelmez. Kubernetes'e hangi imajın kullanılacağı, kaç uygulama kopyasının çalıştırılacağı ve çalışma sırasında hangi ayarlara ihtiyaç duyulduğu bildirilmelidir. Ayrıca imaj, uygulamayı çalıştıracak düğümlerin erişebileceği bir yerde bulunmalıdır. [18]

İmajın hazırlanması ile uygulamanın dağıtılması, birbirini tamamlayan iki ayrı işlemdir. İmaj çalıştırılacak yazılım paketini, Kubernetes nesneleri ise bu paketin küme üzerinde nasıl çalıştırılacağını tanımlar. Aynı imaj farklı çalışma ortamlarında kullanılabilir; ancak dağıtım tanımları hedef ortamın yönetim modeline ve uygulamanın gereksinimlerine göre düzenlenir.

İmajın Pod içinde çalıştırılması

Kubernetes'te konteynerler Pod içinde çalışır. Pod, bir veya birden fazla konteyneri ortak ağ ve depolama kaynaklarıyla bir araya getiren en küçük dağıtım birimidir. Bir Pod içindeki konteynerler aynı düğümde çalıştırılır. Başlangıç düzeyindeki uygulamalarda, bir Pod içinde tek bir uygulama konteyneri bulunması yaygın bir düzenlemedir [5].

Web bileşeni ve PostgreSQL veritabanından oluşan bir sistemde, bu bileşenler ayrı Pod'larda çalıştırılabilir. Böylece çalışma, kaynak ve depolama gereksinimleri ayrı ayrı tanımlanır. Web imajının Pod tanımında belirtilmesi, veritabanı bağlantısının veya kalıcı depolamanın kendiliğinden hazır olduğu anlamına gelmez. Bu ihtiyaçlar ilgili Kubernetes nesneleriyle ayrıca karşılanır.

Podların Deployment ile yönetilmesi

Sürekli çalışan web uygulamasını yalnızca tek bir Pod oluşturarak bırakmak yerine, Deployment (dağıtım yönetimi nesnesi) ile yönetiriz. Deployment, kullanılacak Pod şablonunu ve istenen kopya sayısını tanımlar. Oluşturduğu ReplicaSet (kopya kümesi), bu tanıma uygun Pod sayısını korumaya çalışır. Böylece yönetim, tek tek Podlara verilen işlemlerden uygulamanın istenen durumunun tanımlanmasına taşınır [6].

Örneğin web uygulamasının iki kopyasının çalışması istendiğinde, bu hedef Deployment içinde belirtilir. Yönetilen Podlardan biri silinirse ReplicaSet eksik kopyayı karşılamak üzere yeni bir Pod oluşturur. Yeni Pod, silinen Podun kimliğini sürdürmez; aynı şablona göre oluşturulan başka bir örnektir. Kümenin kaynakları yetersizse veya imaja erişilemiyorsa yeni Podun çalışması gecikebilir. Dolayısıyla istenen durumu tanımlamak, bu durumun hemen gerçekleştiğini garanti etmez [6].

Tablo 4. Web uygulamasının dağıtımında tanımlanan temel bilgiler.

Bilgi	Yanıtladığı soru	Örnek
İmaj	Hangi yazılım sürümü çalıştırılır?	Sürümü belirlenmiş web uygulaması imajı
Pod şablonu	Her uygulama kopyası nasıl çalışır?	Konteyner, çalışma ayarları ve bağlantılar
Kopya sayısı	Kaç Pod çalışması istenir?	İki web Pod'u
Kaynak gereksinimleri	Ne kadar CPU ve RAM ihtiyacı var?	Konteyner için kaynak talepleri ve sınırları
Yapılandırma bağlantıları	Uygulama gerekli ayarları nereden alır?	ConfigMap ve Secret başvuruları

İmaj erişimi ve uygulamanın doğrulanması

Yerel Docker ortamında bulunan bir imajın Kubernetes tarafından otomatik olarak kullanılabildiği varsayılmamalıdır. Seçilen yerel küme aracına göre imajın kümenin kullandığı imaj ortamına yüklenmesi veya bir registry (imaj deposu) üzerinden erişilebilir olması gerekebilir. İmaj adı, sürümü ve indirme davranışı birlikte değerlendirilir [18].

Dağıtımın doğrulanmasında yalnız Kubernetes nesnesinin oluşturulmasına bakılmaz. İstenen Podların oluşması, konteynerlerin başlaması ve uygulamanın hizmet vermeye hazır olması da kontrol edilir. Kullanıcının uygulamaya erişmesi ise ayrıca bir erişim düzenlemesi gerektirir.

Dağıtım süreci; imajın erişilebilir olması, Pod şablonunun doğru tanımlanması ve oluşan uygulama kopyalarının durumunun doğrulanmasıyla birlikte değerlendirilir. İmaj yazılım paketini sağlar; Deployment ise bu paketten oluşturulan Pod'ların istenen sayıda bulunmasını ve sürüm değişikliklerini yönetir.

4.2. Service ile İletişim ve Uygulamaya Erişim

Bir uygulamanın Pod içinde çalışması, kullanıcıların veya diğer uygulama bileşenlerinin ona erişebildiği anlamına gelmez. Web ve veritabanından oluşan bir sistemde iki ayrı

erişim ihtiyacı bulunur: kullanıcıların web uygulamasına ulaşması ve web bileşeninin veritabanına bağlanması. Bu bağlantıların adresleri, portları ve erişim kapsamları ayrı ayrı düzenlenir.

Değişen Podlar için kararlı bir erişim noktası

Podlar yeniden oluşturulduğunda adresleri değişebilir. Bu nedenle uygulama bileşenlerinin bağlantı ayarlarında tek bir Podun adresine bağımlı kalması uygun değildir. Service (servis), belirli bir işlevi sağlayan Podlara kararlı bir erişim noktası sunar. Podlar değişirken istemciler aynı Service üzerinden bağlantı kurmayı sürdürebilir [19].

Service, uygulamayı çalıştıran bir konteyner veya Deployment'ın yerine geçen bir nesne değildir. Deployment uygulamanın çalışma durumunu yönetir; Service ise uygulamaya erişimi düzenler. Bir Service'in oluşturulması, arkasında erişilebilir ve hazır bir uygulama bulunduğunu tek başına kanıtlamaz.

Service'in hangi Podlara yönlendirme yapacağı genellikle label (etiket) ve selector (seçici) ilişkisiyle belirlenir. Podlara verilen etiketlerle Service'in seçicisi eşleşmelidir. Yanlış eşleşme, çalışan Podlar bulunmasına rağmen Service üzerinden uygulamaya ulaşılamamasına neden olabilir [19].

Web bileşeni ile veri tabanının iletişimi

Ayrı Pod'larda çalışan web ve PostgreSQL bileşenleri arasında iletişim sağlamak için veritabanına küme içinden erişilen bir Service oluşturulabilir. Web bileşeni, veritabanı Pod'unun değişebilen adresi yerine bu Service'in adını kullanır. Böylece bağlantı yapılandırması belirli bir Pod adresine bağımlı kalmaz.

Kubernetes'in *Domain Name System (Alan Adı Sistemi, DNS)* hizmeti, Service adlarının çözümlenmesini sağlar. Aynı ad alanındaki bir web Podu, örneğin `db` adlı Service'e kısa adıyla ulaşabilir. Farklı ad alanlarında ise ad alanını da içeren bir adres kullanılır. Bu adlandırma sayesinde bağlantı yapılandırması tek bir Podun adresine bağlı kalmaz [20].

Burada `localhost`, bağlantıyı başlatan Podun kendi ağ ortamını ifade eder. Web ve PostgreSQL ayrı Podlarda çalışıyorsa web uygulamasının veri tabanı adresini localhost olarak belirlemek, PostgreSQL Poduna bağlantı sağlamaz. Veri tabanı Service'inin adı kullanılmalıdır [21].

Küme içi ve küme dışı erişim

Service türü, uygulamaya hangi erişim modelinin sağlandığını belirler. Tablo 5, başlangıç düzeyinde bilinmesi gereken üç türü karşılaştırır [19].

LoadBalancer türünü seçmek, her ortamda otomatik olarak dış erişim adresi oluşmasını sağlamaz. Bunun için altyapının uygun yük dengeleyici desteği bulunmalıdır. NodePort erişimi de düğüm adresinin öğrencinin bilgisayarından ulaşılabilir olmasına bağlıdır [19].

Tablo 5. Temel Service türleri ve erişim amaçları.

Service türü	Erişim modeli	Örnek kullanım
ClusterIP (küme içi IP adresi)	Küme içi bir erişim adresi sağlar; varsayılan türdür.	Web bileşeninin veri tabanına bağlantısı
NodePort (düğüm portu)	Service'i düğüm adresi ve ayrılmış bir port üzerinden erişime açar.	Yerel ortamın ağ yapısına bağlı alternatif
LoadBalancer (yük dengeleyici)	Desteklenen altyapıda harici yük dengeleyiciyle erişim sağlar.	İsteğe bağlı bulut uygulaması

Port yönlendirme ile geçici erişim

Port forwarding (port yönlendirme), yönetim aracının çalıştığı bilgisayardaki bir port ile kümede çalışan bir Pod'un portu arasında geçici bağlantı kurar. Service üzerinden bir hedef seçilerek de kullanılabilir. Bu yöntem, uygulamayı incelemek ve hata ayıklamak için yararlıdır; harici yük dengeleyici oluşturmadan yerel tarayıcıdan erişim sağlayabilir. [22]

Port yönlendirme oturumunun açık kalması gerekir. Oturum kapandığında bu erişim yolu sona erer; uygulamanın küme içinde çalışması ise devam edebilir. Dolayısıyla tarayıcı bağlantısının kesilmesi her zaman uygulamanın durduğu anlamına gelmez [22].

Erişim düzenlenirken bağlantının nereden başlatıldığı, hangi bileşene yöneldiği ve hangi ağ kapsamından kullanılacağı belirlenir. Web uygulamasının kullanıcılar tarafından erişilebilir olması, veritabanının da küme dışına açılmasını gerektirmez. Port yönlendirme ise geçici bir inceleme yoludur; tek başına Service'in normal trafik yönlendirmesini veya birden fazla Pod'a erişim davranışını doğrulamaz.

4.3. Yapılandırma, Gizli Bilgiler ve Kalıcı Veri

Bir uygulamanın çalışması için gereken her bilgi konteyner imajının içine yerleştirilmez. Veri tabanı adresi ve portu çalışma ortamına göre değişebilir; parolalar gizli tutulmalıdır. Kullanıcıların oluşturduğu kayıtların da uygulama yeniden başlatıldığında korunması gerekir. Kubernetes'te bu ihtiyaçları yapılandırma, gizli bilgi ve depolama nesneleriyle ayrı ayrı ele alınır.

Web ve veritabanından oluşan bir sistemde üç bilgi türü ayırt edilir: uygulamanın çalışma ayarları, erişim için kullanılan gizli bilgiler ve veritabanında tutulan kullanıcı verileri. Bunların görevleri ve yaşam süreleri farklıdır. Bu nedenle yapılandırma, gizli bilgi ve kalıcı veri ayrı mekanizmalarla yönetilir.

Çalışma ayarlarının ConfigMap ile sağlanması

ConfigMap (yapılandırma haritası), gizli olmayan yapılandırma bilgilerini anahtar–değer biçiminde saklar. Uygulama bu bilgileri ortam değişkenleri veya dosyalar üzerinden kullanabilir. Böylece çalışma ayarları uygulama imajından ayrılır; bir bağlantı adresini değiştirmek için imajın yeniden oluşturulması gerekmez [23].

Örneğin veritabanı Service'inin adı, bağlantı portu ve veritabanı adı ConfigMap içinde tutulabilir. Pod tanımı, bu değerlerin konteynere ortam değişkeni veya dosya olarak nasıl sağlanacağını belirtir. ConfigMap'in kümede bulunması, uygulamanın onu kendiliğinden okuyacağı anlamına gelmez; uygulama sağlanan değerleri kullanacak şekilde düzenlenmelidir.

Yapılandırma değişikliklerinin çalışan uygulamaya nasıl yansıdığı da önemlidir. ConfigMap değerleri ortam değişkeni olarak kullanılıyorsa mevcut konteynerin ortamı otomatik güncellenmez; yeni değerlerin alınması için Podların yeniden oluşturulması gerekir. Dosya olarak sağlanan değerlerde ise güncellemenin uygulama tarafından yeniden okunması gerekebilir [23].

Gizli bilgilerin Secret ile ayrılması

Secret (gizli bilgi nesnesi), parola ve uygulama anahtarı gibi hassas bilgileri saklamak için kullanılır. Örneğin veritabanı parolası ve oturum güvenliğinde kullanılan uygulama anahtarı bu gruptadır. Bu bilgiler imajın içine veya paylaşılabilir yapılandırma dosyalarına doğrudan yazılmamalıdır. Secret değerleri, ihtiyaç duyan konteynerlere dosya veya ortam değişkeni olarak sağlanabilir. [24]

Secret kullanmak tek başına bütün güvenlik gereksinimlerini karşılamaz. Özellikle Base64 kodlaması şifreleme değildir; kodlanmış bir değer geri çözülebilir. Secret'ın korunması, erişim yetkilerine ve kümenin depolama güvenliği ayarlarına da bağlıdır [24].

Gizli bilgiler yalnızca ihtiyaç duyan bileşenlere sağlanmalı ve erişim yetkileri sınırlandırılmalıdır. Bu bilgilerin günlüklerde, paylaşılan dosyalarda veya ekran görüntülerinde açığa çıkmaması da gözetilmelidir.

Verilerin Pod yaşam süresinden ayrılması

Konteynerin kendi yazılabilir katmanındaki dosyalar kalıcı veri saklamak için yeterli değildir. Konteyner yeniden oluşturulduğunda bu katmandaki veriler kaybolabilir. Kubernetes'in volume mekanizması, verilerin konteynerden ayrı tutulmasını sağlar. Ancak her volume türü aynı kalıcılığı sunmaz: örneğin emptyDir (geçici veri alanı) konteyner yeniden başlatmalarında korunur, fakat bağlı olduğu Pod kaldırıldığında silinir [25].

Bir veritabanı kayıtlarının Pod yeniden oluşturulduğunda da korunabilmesi için kalıcı depolama gerekir. Kubernetes'te bunun karşılanmasında iki nesne birlikte çalışır:

- PersistentVolume (kalıcı veri alanı, PV): Kümede kullanılabilir depolama kaynağını temsil eder.
- PersistentVolumeClaim (kalıcı veri alanı talebi, PVC): Uygulamanın kapasite ve erişim biçimi gibi depolama ihtiyaçlarını belirtir.

Pod, PVC'ye başvurarak kendisine ayrılan depolama alanını kullanır. Böylece veri alanının yaşam süresi tek bir Podun yaşam süresinden ayrılır [26].

StorageClass (depolama sınıfı) ise depolamanın hangi yöntem ve özelliklerle sağlanacağını tanımlar. Uygun destek bulunduğunda PVC talebine karşılık depolama alanı dinamik olarak oluşturulabilir. Yerel ortamla bulut ortamında aynı kavramlar kullanılsa da depolama sınıfları ve bunların dayandığı altyapı farklı olabilir [27].

Tablo 6. Uygulama bilgilerinin türleri ve Kubernetes karşılıkları.

Bilgi türü	Uygulamadaki örnek	Kubernetes karşılığı
Gizli olmayan çalışma ayarı	Veri tabanı Service adı ve portu	ConfigMap
Hassas erişim bilgisi	Veri tabanı parolası ve uygulama anahtarı	Secret
Kullanıcının oluşturduğu kalıcı veri	Ders notları ve kullanıcı kayıtları	PVC üzerinden kullanılan PV

Kalıcılığın doğrulanması ve sınırları

Veri kalıcılığı, Pod yeniden oluşturulduğunda aynı depolama alanındaki kayıtlara erişilebilmesiyle değerlendirilebilir. Bu değerlendirmede aynı PVC ve veri alanı korunmalıdır. PVC'nin silinmesi ise farklı bir işlemdir: PV'nin reclaim policy (geri kazanım politikası), depolamanın korunmasına veya silinmesine yol açabilir. [26]

Kalıcı depolama, yedekleme ile aynı şey değildir. Pod'un değiştirilmesine dayanabilen veri, depolama kaynağının kaybına veya yanlışlıkla silinmesine karşı ayrıca korunmalıdır. Benzer biçimde, Docker volume içinde bulunan veriler aynı uygulama imajı Kubernetes üzerinde çalıştırıldığında kendiliğinden aktarılmaz. İmaj yazılım paketini taşır; mevcut verinin aktarılması ayrı bir işlemdir.

5. Uygulamanın Sağlıklı Çalışması ve Ölçeklenmesi

Önceki bölümde uygulamanın Kubernetes üzerinde çalıştırılmasını, bileşenler arasındaki iletişimi ve veri gereksinimlerini ele aldık. Ancak uygulamanın başlatılması, hizmet vermeye hazır olduğunu veya çalışması boyunca sağlıklı kalacağını tek başına göstermez. Uygulamanın başlangıç sürecini, isteklere cevap verebilmesini ve hata durumundaki davranışını da değerlendirmek gerekir.

Bu bölümde sağlık kontrolleri ve hata durumlarında Kubernetes'in verdiği tepkiler açıklanır. Ardından işlemci ve bellek gereksinimlerinin tanımlanması, uygulama kopyalarının artırılıp azaltılması ve sürüm değişikliklerinin yönetilmesi ele alınır. Bu mekanizmaların hangi ihtiyaçları karşıladığı ve hangi sorunları tek başına çözemediği örneklerle değerlendirilir.

5.1. Sağlık Kontrolleri ve Hata Davranışı

Bir konteynerin çalışıyor olması, uygulamanın kullanıcı isteklerini karşılayabildiğini göstermez. Uygulama başlangıç işlemlerini sürdürüyor, veri tabanına bağlanamıyor veya çalışmasına rağmen cevap üretemiyor olabilir. Kubernetes'in uygun tepkiyi verebilmesi için bu durumların birbirinden ayrılması gerekir.

Probe (sağlık kontrolü), kubelet'in konteynerdeki uygulamayı belirli aralıklarla sınamasını sağlar. Kontrolün sonucu, yalnız uygulamanın durumunu bildirmek için değil, trafiğe kabul edilmesi veya yeniden başlatılması gibi kararlar için de kullanılır. Kubernetes üç temel sağlık kontrolü sunar [28].

Başlatma, hazır olma ve canlılık kontrolleri.

Başlatma kontrolü tanımlanmışsa başarılı olana kadar hazır olma ve canlılık kontrolleri çalıştırılmaz. Bu düzen, başlangıcı uzun süren bir uygulamanın henüz hazırlanırken hatalı kabul edilmesini önler. Hazır olma kontrolünün başarısızlığı ise tek başına konteyneri yeniden başlatmaz [28].

Tablo 7. Kubernetes sağlık kontrollerinin amaçları ve başarısızlık sonuçları.

Kontrol	Yanıtladığı soru	Başarısızlık durumundaki davranış
Startup probe (başlatma kontrolü)	Uygulama başlangıç işlemlerini tamamladı mı?	Tanımlanan başarısızlık eşiğine ulaşıldığında konteyner sonlandırılır; yeniden başlatma politikası uygulanır.
Readiness probe (hazır olma kontrolü)	Uygulama şu anda istek kabul etmeye hazır mı?	Pod hazır değil olarak işaretlenir; normal Service trafiğinin hedefleri arasından çıkarılır.
Liveness probe (canlılık kontrolü)	Uygulama çalışmasını sürdürebiliyor mu, yoksa yeniden başlatılmalı mı?	Tanımlanan başarısızlık eşiğine ulaşıldığında ilgili konteyner yeniden başlatılır.

Kontroller, uygulamanın sunduğu bir web adresine istek gönderilmesi veya konteyner içinde bir komutun sonucunun değerlendirilmesi gibi yöntemlerle gerçekleştirilebilir. Kontrol aralığı, cevap için beklenecek süre ve başarısızlık eşiği uygulamanın davranışına uygun seçilmelidir. Çok kısa süreler normal gecikmeleri hata gibi değerlendirebilir; çok uzun süreler gerçek sorunun fark edilmesini geciktirebilir [29].

Veritabanı kesintisi örneği

Bir web uygulamasının kayıtları okuyup yazabilmek için PostgreSQL veritabanına ihtiyaç duyduğunu düşünelim. Uygulama, veritabanına bağlanamadığında temel işlevlerini yerine getiremiyorsa hazır olma kontrolü bu bağımlılığı da değerlendirebilir. Yalnızca web sürecinin yanıt vermesi, uygulamanın kullanıcı isteklerini karşılamaya hazır olduğunu göstermeyebilir.

PostgreSQL geçici olarak erişilemez olduğunda web süreci çalışmaya devam edebilir. Bu senaryoda hazır olma kontrolünün başarısız olması, web Podunun normal Service trafiğini kabul etmesini durdurur. Veri tabanı bağlantısı geri geldiğinde ve kontrol başarılı olduğunda Pod yeniden trafiğe kabul edilebilir [28].

Canlılık kontrolü, uygulamanın kendi işleyişini sürdürebilip sürdüremediğini ve yeniden başlatılmasının gerekip gerekmediğini değerlendirir. Veritabanının geçici kesintisi, web

konteynerinin mutlaka yeniden başlatılması gereken bir arıza olarak yorumlanmamalıdır. Web konteynerini tekrar tekrar başlatmak veritabanı sorununu çözmez; toparlanmayı da zorlaştırabilir. Bu nedenle hazır olma ve canlılık kontrolleri farklı koşulları sınamalıdır.

Konteynerin yeniden başlatılması ve Podun yenilenmesi

Konteynerin yeniden başlatılması ile Pod'un yeniden oluşturulması aynı işlem değildir. kubelet, yeniden başlatma politikasına göre bir konteyneri aynı Pod içinde yeniden başlatabilir. Deployment tarafından yönetilen bir Pod kaldırıldığında ise eksik kopyayı karşılamak üzere yeni bir Pod oluşturulur. Hata durumunu değerlendirirken konteynerin yeniden başlatılıp başlatılmadığı ile Pod'un kimliğinin değişip değişmediği ayrı incelenir. [6], [30]

Bir konteyner sürekli başarısız olup yeniden başlatılıyorsa *CrashLoopBackOff* (tekrarlayan çökme nedeniyle bekleme) durumu görülebilir. Bu ifade tek başına hatanın nedenini açıklamaz; yeniden başlatmalar arasında bekleme uygulandığını gösterir. Hatalı yapılandırma, uygulama hatası veya başarısız sağlık kontrolleri gibi nedenler ayrıca incelenmelidir [30].

Sağlık kontrollerinin doğrulanması

Sağlık kontrolü, yalnız sınadığı koşul hakkında bilgi verir. Basit bir cevap kontrolünün başarılı olması; kullanıcı girişinin, kayıt eklemenin ve veri kalıcılığının tümüyle doğru çalıştığını kanıtlamaz. Bu nedenle sağlık kontrolleri, uygulamanın işlevsel doğrulamalarını tamamlar.

Sağlık davranışı değerlendirilirken Pod'un hazır olma durumu, konteynerin yeniden başlatma sayısı ve Service'in trafik yönlendirebildiği hedefler birlikte incelenir. Bu bilgiler, hazır olma durumunun değişmesi ile konteynerin yeniden başlatılmasını ayırt etmeyi sağlar. Yerel port yönlendirmesiyle tarayıcıdan erişim ise Service'in normal trafik davranışını tek başına doğrulamaz.

5.2. Kaynak Gereksinimleri ve Ölçekleme

Uygulama kopyalarının çalışabilmesi için kümede yeterli işlemci ve bellek bulunmalıdır. Kopya sayısını artırmak, mevcut kaynak kapasitesini artırmaz; aynı kapasiteyi paylaşan iş yüklerinin sayısını artırır. Bu nedenle ölçekleme, uygulamanın kaynak gereksinimleriyle birlikte değerlendirilir.

Kaynak talepleri ve kullanım sınırları

Kubernetes'te konteynerlerin işlemci ve bellek gereksinimleri iki temel değerle tanımlanır:

- Request (kaynak talebi): Podun uygun bir düğüme yerleştirilmesinde dikkate alınan kaynak miktarıdır.
- Limit (kullanım sınırı): Konteynerin kaynak kullanımına uygulanan üst sınırdır.

Scheduler (zamanlayıcı), Pod için uygun düğümü seçerken kaynak taleplerini ve düğümlerin kullanılabilir kapasitesini değerlendirir. Gereksinimleri karşılayan bir düğüm bulunamazsa Pod yerleştirilemez ve Pending durumunda kalabilir. Kaynak talebi, uygulamanın her an tam olarak o miktarı tükettiği anlamına gelmez [31].

İşlemci miktarı *Central Processing Unit (merkezi işlem birimi, CPU)* birimiyle belirtilir. 1000m, bir CPU birimine; 500m, yarım CPU birimine karşılık gelir. Buradaki m, birimin binde birini gösterir. Bellek için kullanılan Mi ise mebibyte (mebibayt, MiB) birimini ifade eder [31].

Tablo 8. Tek web konteyneri için örnek kaynak talepleri ve sınırları.

Kaynak	Talep	Sınır	Anlamı
CPU	100m	500m	Yerleştirmede 0,1 CPU hesaba katılır; kullanım 0,5 CPU ile sınırlandırılır.
Bellek	128Mi	256Mi	Yerleştirmede 128 MiB hesaba katılır; bellek sınırı 256 MiB'dir.

Bu değerler kavramları açıklamak için verilmiş örneklerdir; bütün web uygulamaları için uygun kabul edilmemelidir. Kaynak talepleri ve sınırları, uygulamanın başlangıç ve yük altındaki davranışı gözlemlenerek belirlenir.

İşlemci ve bellek sınırlarının farklı etkileri

CPU sınırına ulaşan bir konteynerin işlemci kullanımı kısıtlanabilir. *CPU throttling (işlemci kullanımının kısıtlanması)*, uygulamanın cevap sürelerini uzatabilir; tek başına konteynerin sonlandırılması anlamına gelmez [32].

Bellek sınırını aşmaya çalışan bir konteyner ise bellek yetersizliği nedeniyle sonlandırılabilir. Bu durum *Out Of Memory (bellek yetersizliği, OOM)* ile ilişkilidir ve sonlandırma nedeni olarak OOMKilled görülebilir. Yeniden başlatma, uygulamanın bellek ihtiyacıyla sınır arasındaki uyumsuzluğu kendiliğinden çözmez [33].

Bu nedenle kaynak sınırlarını çok düşük belirlemek de gereğinden fazla talepte bulunmak da sorun oluşturabilir. İlk durumda uygulama yavaşlayabilir veya sonlandırılabilir; ikinci durumda kümede yeni Podların yerleştirilmesi zorlaşabilir.

Uygulama kopyalarının artırılması ve azaltılması

Horizontal scaling (yatay ölçekleme), uygulama kopyalarının sayısını değiştirmektir. *Vertical scaling (dikey ölçekleme)* ise bir kopyaya ayrılan işlemci ve bellek miktarının değiştirilmesidir. Kubernetes'te yatay ölçekleme, örneğin Deployment'ın istenen Pod sayısının artırılmasıyla gerçekleştirilir [6], [34].

Örneğin bir web Deployment'ının kopya sayısı birden ikiye çıkarıldığında, aynı Pod şablonundan ikinci bir web Pod'u oluşturulur. Service, seçicisine uyan ve kullanıma hazır olan kopyalara trafik yönlendirir. Kopya sayısı tekrar bire indirildiğinde fazla Pod kaldırılır. [6], [19]

Tablo 8'deki değerlere sahip, tek konteynerli iki web Pod'unun toplam kaynak talebi 200m CPU ve 256 MiB bellektir. Bu hesap yalnız web bileşenini kapsar; veritabanı ve kümenin sistem bileşenleri de kaynak kullanır. İkinci Pod'un yerleştirilebilmesi için uygun bir düğümde yeterli kapasite bulunmalıdır. Bu nedenle istenen kopya sayısı ile gerçekten hazır olan kopya sayısı ayrı değerlendirilir.

Ölçeklemenin sınırları ve otomatik ölçekleme

Web kopyalarını artırmak, uygulamanın kapasitesinin aynı oranda artacağını garanti etmez. Veritabanı darboğazı, bağlantı sınırları veya ortak veriye erişim biçimi sonucu etkileyebilir. Web bileşeninin ölçeklenmesi, veritabanını kendiliğinden ölçeklemez. Benzer biçimde, PostgreSQL Pod sayısını artırmak tek başına tutarlı bir veritabanı çoğaltma düzeni oluşturmaz; veritabanının ölçeklenmesi ayrıca tasarlanmalıdır.

Horizontal Pod Autoscaler (yatay Pod otomatik ölçekleyicisi, HPA), ölçümlere göre kopya sayısını otomatik değiştirebilir. CPU kullanımına göre kaynak talepleri doğru tanımlanmalı ve gerekli ölçüm hizmeti bulunmalıdır. Dolayısıyla Kubernetes kurulmuş olması, otomatik ölçeklemenin hazır olduğu anlamına gelmez [34].

5.3. Uygulamanın Güncellenmesi ve Önceki Sürüme Dönüş

Uygulamaya yeni bir özellik eklendiğinde veya bir hata düzeltildiğinde yeni bir konteyner imajı hazırlanır. Kubernetes üzerinde sürüm değişimi, çalışan konteynerin içine dosya kopyalamak yerine Deployment'ın Pod şablonunda kullanılacak imajın değiştirilmesiyle gerçekleştirilir. Bu değişiklik yeni bir dağıtım süreci başlatır [6].

Örneğin bir web uygulamasının 1.1 sürümünden 1.2 sürümüne geçişinde, Deployment'ın Pod şablonundaki imaj bilgisi güncellenir. Amaç yalnızca yeni sürümü çalıştırmak değil, geçişin ilerleyişini ve uygulamanın hizmet verebilmesini izlemek, gerektiğinde önceki sürüme dönebilmektir.

Güncellenecek sürümün açıkça belirlenmesi

Kullanılan imaj sürümü izlenebilir olmalıdır. Aynı etiketi farklı imaj içerikleri için tekrar kullanmak, çalışan sürümün anlaşılmasını zorlaştırır. Özellikle latest etiketi belirli ve değişmez bir sürümü ifade etmez. Anlamlı sürüm etiketleri kullanmak ve gerektiğinde imajı digest ile kesin olarak belirtmek, dağıtımın tekrarlanabilirliğini artırır [18].

Ayrıca bir imaj deposundaki etiketin içeriğini değiştirmek, mevcut Deployment'ın Pod şablonunu kendiliğinden değiştirmez. Yeni dağıtımın hangi imajı kullanacağı açıkça tanımlanmalıdır. Önceki sürümün imajı da geri dönüş gerektiğinde erişilebilir olmalıdır.

Kademeli güncelleme

Rolling update (kademeli güncelleme), eski sürümdeki Podların kontrollü biçimde yeni sürümdeki Podlarla değiştirilmesidir. Yeni kopyalar oluşturulurken eski kopyalar aşamalı olarak azaltılır. Deployment'ın varsayılan güncelleme stratejisi bu yaklaşımdır [6], [35].

Geçiş sırasında kaç ek Pod oluşturulabileceği ve kaç kopyanın kullanılamaz olabileceği sınırlandırılır. Böylece güncellemenin kaynak ihtiyacı ve hizmet kapasitesine etkisi düzenlenir. Eski ve yeni sürümün bir süre birlikte çalışabilmesi, uygulama sürümlerinin ortak veri ve bağlantı düzeniyle uyumlu olmasını gerektirir [35].

Tablo 9. Temel Deployment güncelleme stratejileri.

Strateji	Geçiş biçimi	Temel etkisi
RollingUpdate (kademeli güncelleme)	Yeni Podlar oluşturulurken eski Podlar aşamalı kaldırılır.	Hizmet kapasitesini korumaya çalışır; geçici ek kaynak gerektirebilir.
Recreate (yeniden oluşturma)	Eski sürümün Podları sonlandırıldıktan sonra yeni sürüm başlatılır.	Geçiş sırasında hizmet kesintisi oluşabilir.

Kademeli güncelleme, tek başına kesintisizlik garantisi değildir. Yeni Podların hazır olması, yeterli kaynak bulunması ve uygulamanın geçişe uygun davranması gerekir. Hazır olma kontrolleri, yeni kopyaların trafiğe kabul edilmesini düzenleyerek bu süreci destekler [35].

Güncellemenin doğrulanması

Yeni sürüme geçerken Deployment'ın ilerleme durumu ve hazır kopya sayısı izlenir. Yanlış imaj adı, imaja erişim sorunu veya başarısız sağlık kontrolü güncellemenin tamamlanmasını engelleyebilir. Kubernetes'in ilerleme sorunu bildirmesi, otomatik olarak önceki sürüme döndüğü anlamına gelmez [6].

Güncellemenin doğrulanmasında üç temel soru değerlendirilir:

- İstenen yeni sürümü kullanan Pod'lar oluştu ve kullanıma hazır hâle geldi mi?
- Çalışan uygulamanın yeni sürümü kullandığı doğrulanabiliyor mu?
- Uygulamanın temel işlevleri beklenen sonuçları üretiyor mu?

Dağıtımın tamamlanması, uygulamanın bütün işlevlerinin hatasız olduğunu kanıtlamaz. Bu nedenle Kubernetes durum bilgileriyle uygulama düzeyindeki kontroller birlikte değerlendirilir.

Önceki sürüme dönmenin kapsamı

Rollback (önceki sürüme dönüş), saklanan Deployment revizyonlarından birinin Pod şablonuna dönülmesini sağlar. Ancak bu işlem uygulamanın bütün verilerini ve dış yapılandırmalarını geçmişteki hâline getirmez [6].

Önceki imaja dönülmesi, veritabanında sonradan oluşturulan kayıtları veya yapılan şema değişikliklerini geri almaz. Benzer biçimde, aynı adla kullanılan bir ConfigMap'in içeriği değiştirildiyse Deployment revizyonuna dönmek o nesnenin eski içeriğini kendiliğinden

geri getirmez. Uygulama sürümü, dış yapılandırma ve kalıcı veri ayrı değerlendirilmelidir. [6], [23]

Geri dönüşün başarılı olması için önceki imajın erişilebilir olması ve bu sürümün mevcut yapılandırma ile veritabanı yapısıyla uyumlu çalışabilmesi gerekir. Bu nedenle sürüm değişikliği planlanırken yalnız yeni sürüme geçiş değil, önceki sürüme dönüşün koşulları da değerlendirilir.

6. Bulut Ortamında Kubernetes

Kubernetes'in dağıtım, iletişim, yapılandırma, depolama ve uygulama yönetimi kavramları bulut ortamında da geçerlidir. Ancak imajın taşınabilir olması; ağ erişiminin, depolamanın ve erişim yetkilerinin her ortamda değişmeden kullanılabileceği anlamına gelmez. Bu gereksinimler, hedef kümenin altyapısına ve sunduğu hizmetlere göre değerlendirilir.

Bu bölümde yerel kümeler ile yönetilen Kubernetes hizmetlerinin farkları, kullanıcı ve sağlayıcı arasındaki sorumluluk paylaşımı ve uygulamaların bulut ortamına uyarlanması ele alınır. AWS, Microsoft Azure ve Google Cloud örnekleri ortak Kubernetes kavramları üzerinden karşılaştırılır. Ayrıca uygulamanın çalışmasının doğrulanması ve kullanılmayan kaynakların kaldırılması açıklanır.

6.1. Yerel Küme ve Yönetilen Kubernetes Hizmetleri

Yerel Kubernetes kümesi, uygulamanın dağıtımını ve yönetimini kişisel bilgisayar üzerinde öğrenmek için uygun bir ortamdır. Bu ortamda Pod, Deployment ve Service gibi nesnelerin davranışları gözlemlenebilir; uygulamanın sağlık kontrolleri, ölçeklenmesi ve güncellenmesi denenebilir. Ancak kullanılabilir işlemci, bellek ve depolama kapasitesi yerel bilgisayarın kaynaklarıyla sınırlıdır. Bilgisayarın kapatılması veya yerel Kubernetes ortamının durdurulması, uygulamaya erişimi de keser.

Bulutta Kubernetes kullanıldığında kümenin çalıştığı altyapı, bulut sağlayıcısının kaynaklarından oluşturulur. Bu altyapıda Kubernetes kurulumu ve işletimi kullanıcı tarafından üstlenilebilir veya managed Kubernetes (yönetilen Kubernetes) hizmetlerinden yararlanılabilir. Yönetilen hizmetlerde sağlayıcı, kontrol düzleminin işletimini üstlenir. Çalışan düğümlerin yönetiminde ise kullanıcı ile sağlayıcı arasındaki sorumluluk dağılımı, seçilen hizmet modeline göre değişir. [36]–[38]

Yönetilen hizmetlerde sorumlulukların paylaşılması

Kubernetes kontrol düzleminin yönetilmesi; kümenin durumunu tutan ve uygulama nesnelerini yöneten bileşenlerin işletilmesini kapsar. Yönetilen hizmetler, bu bileşenlerin bakım yükünü azaltır. Bazı hizmet seçenekleri çalışan düğümlerin oluşturulması, ölçeklenmesi ve güncellenmesi gibi işlemleri de otomatikleştirir. Bununla birlikte, altyapının yönetilmesi ile uygulamanın yönetilmesi farklı sorumluluklardır. Uygulamayı

işleten kullanıcı veya ekip; kullanılacak imajı, uygulama yapılandırmasını, kaynak gereksinimlerini, sağlık kontrollerini ve erişim kurallarını belirlemeye devam eder. [36]–[38]

Tablo 10. Yerel Kubernetes ortamı ile buluttaki yönetilen Kubernetes hizmetlerinin temel farkları.

Karşılaştırma alanı	Yerel Kubernetes ortamı	Bulutta yönetilen Kubernetes
Çalışma altyapısı	Kişisel bilgisayarın kaynaklarını kullanır.	Bulut altyapısındaki kaynakları kullanır.
Kontrol düzlemi	Yerel Kubernetes yazılımı tarafından çalıştırılır; ortamı kullanıcı başlatır ve durdurur.	Sağlayıcı tarafından işletilir.
Çalışan düğümler	Kapasite yerel bilgisayarla sınırlıdır.	Kapasite ve yönetim seçenekleri hizmet modeline göre belirlenir.
Uygulama tanımları	Deployment, Service ve diğer nesneler kullanıcı tarafından hazırlanır.	Aynı Kubernetes nesneleri kullanılır; ortama bağlı ayarlar uyarlanır.
Ağ ve depolama	Yerel ortamın sunduğu olanaklara dayanır.	Bulut ağları ve depolama hizmetleriyle bütünleşir.
Kaynak tüketimi	Yerel bilgisayarın kapasitesini tüketir.	Kullanılan hizmet ve kaynaklara göre ücret oluşabilir.

Tablodaki bulut hizmeti özellikleri, sağlayıcıların resmî açıklamalarına dayanır. [36]–[38]

AWS, Azure ve Google Cloud örnekleri

Üç sağlayıcı da Kubernetes'in temel nesneleriyle çalışmayı sürdürürken altyapı yönetimini kolaylaştıran hizmetler sunar:

- Amazon Elastic Kubernetes Service (Amazon Esnek Kubernetes Hizmeti, EKS): AWS, Kubernetes kontrol düzlemini yönetir. Çalışan düğümlerin yönetimi seçilen yapılandırmaya bağlıdır; EKS Auto Mode, düğüm yönetimindeki otomasyonu genişletir. [36]
- Azure Kubernetes Service (Azure Kubernetes Hizmeti, AKS): Microsoft Azure, kontrol düzlemini yönetir. AKS Standard, düğüm yapılandırmasında kullanıcıya daha fazla seçim sunarken AKS Automatic, altyapı yönetiminin daha büyük bölümünü otomatikleştirir. [37]
- Google Kubernetes Engine (Google Kubernetes Motoru, GKE): Google Cloud, kontrol düzlemini yönetir. Standard modunda düğüm yapılandırması üzerinde daha fazla kullanıcı kontrolü bulunur; Autopilot modunda çalışan düğümlerin yönetimini de sağlayıcı üstlenir. [38]

Bu seçenekler, sağlayıcı seçiminin yanında hangi yönetim sorumluluklarının kullanıcıda kalacağının da değerlendirilmesi gerektiğini gösterir.

Yönetilen Kubernetes hizmetleri, altyapının işletim yükünü azaltır; ancak uygulamanın doğru çalışmasını kendiliğinden garanti etmez. Kontrol düzleminin sağlayıcı tarafından yönetilmesi, uygulamanın bağlantı ayarlarının doğru olduğu veya verilerinin yedeklendiği anlamına gelmez. Bu nedenle hizmet seçilirken sağlayıcının üstlendiği görevlerle kullanıcıda kalan uygulama ve veri yönetimi sorumlulukları birlikte değerlendirilir. [36]–[38]

6.2. Uygulamanın Bulut Ortamına Uyarlanması

Bir Kubernetes uygulamasını buluta taşımak, yerel ortamda kullanılan nesne tanımlarını başka bir kümede çalıştırmaktan daha fazlasını gerektirir. Uygulamanın bileşenleri ve bu bileşenler arasındaki ilişkiler korunabilir; ancak imaj erişimi, yapılandırma, depolama ve dış erişim ayarları hedef ortamın olanaklarına göre değerlendirilir. Taşınabilirlik, bütün ayarların değişmeden kullanılabilmesi değil, uygulamanın temel yapısı korunarak farklı ortamlarda çalıştırılabilmesidir.

Örneğin web ve veritabanından oluşan bir uygulama buluta taşındığında, bileşenlerin görevleri ve aralarındaki temel ilişki korunabilir. Uyarlamanın amacı, bu bileşenlerin çalışma, iletişim ve depolama gereksinimlerini hedef kümenin kaynaklarıyla karşılamaktır.

İmajın küme tarafından erişilebilir olması

Yerel bilgisayarda oluşturulan bir imaj, buluttaki Kubernetes düğümlerinde kendiliğinden bulunmaz. Bu nedenle imajın, kümenin erişebildiği bir imaj kayıt deposunda bulunması gerekir. Deployment tanımındaki imaj adresi de bu konumu göstermelidir. Depo özel erişime sahipse, imajı indirecek düğümlerin gerekli erişim yetkileri sağlanır. Kullanıcının kendi bilgisayarından depoya erişebilmesi, buluttaki kümenin de gerekli ağ erişimine ve indirme yetkilerine sahip olduğunu göstermez. Kümenin imajı indirebilmesi ayrıca doğrulanmalıdır. [18]

Ayrıca imajın hedef düğümlerin işlemci mimarisiyle uyumlu olması gerekir. Böylece buluta geçişte hem imajın bulunabilirliği hem de çalıştırılabilirliği değerlendirilir.

Yapılandırmanın ve gizli bilgilerin uyarlanması

Uygulama ayarlarının imajdan ayrılması, aynı imajın farklı ortamlarda kullanılmasını kolaylaştırır. ConfigMap ile sağlanan ayarlar ve Secret ile sağlanan gizli bilgiler, hedef kümede de oluşturulur. Bu nesneler yerel kümeden bulut kümesine otomatik olarak aktarılmaz. Veritabanı bağlantı bilgileri ve uygulama sırları, bulut ortamı için uygun değerlerle tanımlanır. [23], [24]

Web bileşeni ile PostgreSQL aynı kümede tutulduğunda, veritabanına Service adı üzerinden erişme yaklaşımı korunabilir. Buna karşılık bilgisayara özgü dosya yolları veya yerel erişim adresleri doğrudan bulut ortamına taşınamaz.

Kalıcı depolamanın ve verinin değerlendirilmesi

PostgreSQL'in kalıcı depolama ihtiyacı bulutta da devam eder. PersistentVolumeClaim ile ifade edilen kapasite ve erişim gereksinimleri korunabilir; ancak bu isteği karşılayan StorageClass ve fiziksel depolama altyapısı değişebilir. Yerelde kullanılan depolama sınıfının bulut kümesinde aynı adla ve aynı özelliklerle bulunacağı varsayılmaz. [26], [27]

Uygulamanın dağıtılması ile mevcut verinin taşınması ayrı işlemlerdir. Bulutta yeni bir kalıcı depolama alanı oluşturulması, yerel veritabanı kayıtlarının bu alana aktarılması anlamına gelmez. Mevcut kayıtların hedef ortamda kullanılabilmesi için ayrıca veri aktarımı gerekir. Yeni verilerle başlatılan bir uygulamada ise önceki kayıtların bulunmaması, tek başına depolama hatası olarak yorumlanmamalıdır.

Uygulamanın dış erişiminin düzenlenmesi

Yerel çalışmada kullanılan geçici erişim yöntemleri, buluttaki uygulamanın erişim düzeni için yeniden değerlendirilir. Sağlayıcı bütünleşmesi bulunan bir kümede LoadBalancer türündeki Service, bulut yük dengeleyicisi üzerinden erişim sağlayabilir. Bununla birlikte yük dengeleyicisinin dışa açık mı yoksa özel ağ içinde mi olacağı, ortamın yapılandırmasına bağlıdır. [19]

Web ve veritabanı aynı kümede çalışıyorsa, kullanıcı erişimi web bileşeni için düzenlenebilir. Veritabanına ise web bileşeninin küme içinden erişmesi yeterli olabilir. Uygulamayı bulutta çalıştırmak, bütün bileşenlerin doğrudan internete açılmasını gerektirmez.

Tablo 11. Bir uygulamanın buluta uyarlanmasında korunan yapı ve değerlendirilen ayarlar.

Alan	Korunabilen yapı	Bulut ortamında değerlendirilen ayar
Uygulama bileşenleri	Web ve veritabanı bileşenlerinin ayrımı	Bileşenlerin hedef kümede çalıştırılması
İmaj	Uygulamanın paketlenmiş sürümü	Depo adresi, erişim yetkisi ve mimari uyumu
Yapılandırma	ConfigMap ve Secret kullanımı	Ortama özgü değerler ve gizli bilgiler
Depolama	PostgreSQL için kalıcılık gereksinimi	StorageClass, kapasite ve veri aktarımı
Ağ erişimi	Service üzerinden iletişim	Web bileşeninin dış erişimi
Kaynak yönetimi	Kaynak istekleri, sınırlar ve sağlık kontrolleri	Hedef kümenin kapasitesine uygun değerler

Bu değerlendirme EKS, AKS ve GKE için aynı Kubernetes kavramları üzerinden yapılır. Kullanılan altyapı ve hizmet ayarları sağlayıcıya göre değişse de temel sorular aynıdır: Küme imaja erişebiliyor mu, uygulama doğru yapılandırmayı kullanıyor mu, gerekli bağlantılar kurulabiliyor mu ve veriler uygun depolamada saklanıyor mu?

6.3. Bulutta Doğrulama ve Kaynak Yönetimi

Uygulamanın bulut kümesine dağıtılması, çalışmanın tamamlandığını tek başına göstermez. Kubernetes nesnelerinin oluşturulmasının ardından uygulamanın kullanıcı isteklerini karşılayabildiği, bileşenlerin iletişim kurabildiği ve verilerin beklenen biçimde saklandığı doğrulanır. Çalışmanın sonunda ise korunması gereken veriler belirlenir ve kullanılmayan kaynaklar kaldırılır.

Uygulamanın çalıştığının doğrulanması

Doğrulama, hem Kubernetes nesnelerinin durumunu hem de uygulamanın işlevlerini kapsar. Bir Pod'un Running durumunda olması, uygulamanın kullanıma hazır olduğunu veya bütün işlevlerinin doğru çalıştığını göstermez. Konteyner çalışırken veritabanı bağlantısı başarısız olabilir ya da uygulama kullanıcı isteklerine yanıt veremeyebilir. Bu nedenle Pod durumu, hazır olma kontrolleri ve uygulama üzerinden yapılan işlevsel kontroller birlikte değerlendirilir. [30]

Web ve veritabanından oluşan bir sistemde, web sayfasına erişilebilmesi yalnızca erişim yolunun ve ilgili web işlevinin çalıştığını gösterir. Bir kaydın oluşturulup yeniden okunması, bileşenler arası iletişimi ve veri işlemlerini de sınar. Veritabanı Pod'u aynı kalıcı depolamayla yeniden oluşturulduğunda kayıtların korunması ise veri kalıcılığını değerlendirmeye yardımcı olur. Bu kontrol, yedekleme veya bütün arızalara karşı dayanıklılık testi yerine geçmez.

Tablo 12. Bulutta çalışan bir uygulamanın temel doğrulama ölçütleri.

Kontrol alanı	Beklenen sonuç	Örnek doğrulama
Dağıtım durumu	Beklenen uygulama kopyaları çalışır ve kullanıma hazırdır.	İstenen ve hazır Pod sayılarının karşılaştırılması
Kullanıcı erişimi	Belirlenen erişim noktası beklenen yanıtı verir.	Web arayüzüne erişilmesi
Bileşenler arası iletişim	Uygulama bağımlı olduğu bileşenlerle iletişim kurar.	Veritabanına kayıt yazılıp okunması
Veri kalıcılığı	Pod'un yeniden oluşturulması gerekli verileri kaybettirmez.	Aynı depolamayla önceki kayıtlara erişilmesi
Uygulama kararlılığı	Tekrarlanan hatalar veya sürekli yeniden başlatmalar görülmez.	Durum bilgilerinin, yeniden başlatma sayılarının ve günlüklerin incelenmesi

Bir sorun görüldüğünde bu ölçütler, incelemenin hangi bileşene yönelmesi gerektiğini de gösterir. Örneğin web sayfasının açılması fakat kayıt eklenememesi, dış erişimin çalıştığını; veritabanı bağlantısının veya uygulama yapılandırmasının ayrıca incelenmesi gerektiğini düşündürür.

Uygulamanın durdurulması ile kaynakların kaldırılması

Uygulamanın artık kullanılmaması, ona ayrılan bulut kaynaklarının da kaldırıldığı anlamına gelmez. Web bileşeninin kopya sayısını sıfıra indirmek veya Deployment'ı silmek; kümenin, depolama alanlarının ve ilişkili diğer kaynakların tamamını ortadan kaldırmaz. Kaynakların yaşam döngüleri ayrı değerlendirilir.

Sağlayıcıların küme silme davranışları da farklıdır. AWS, EKS kümesi silinmeden önce ilgili Service kaynaklarının kaldırılmasını, yük dengeleyicilerinin doğru biçimde serbest bırakılması açısından belirtir. Azure'da AKS kümesinin silinmesi, düğüm kaynak grubundaki kaynakları da kapsar. Google Cloud ise GKE kümesi silindiğinde kalıcı disklerin korunabildiğini açıklar. Bu nedenle kümenin silinmesi, bütün ilişkili kaynakların kaldırıldığı şeklinde yorumlanmaz. [39]–[41]

Verilerin korunması ve maliyet kontrolü

Depolama kaynakları kaldırılmadan önce hangi kayıtların saklanacağı belirlenir. Kubernetes'te *reclaim policy (geri kazanım politikası)*, depolama isteği kaldırıldıktan sonra kalıcı depolamanın nasıl ele alınacağını belirler. *Retain* seçeneğinde depolama korunarak ayrıca yönetilir; *Delete* seçeneğinde, desteklenen altyapıda depolama kaynağı da silinir. Bu nedenle bir PersistentVolumeClaim'i kaldırmanın veriler üzerindeki sonucu, ilgili politikaya bağlıdır. [26]

Kaynaklar kaldırılırken uygulamaya ayrılan kaynaklarla başka uygulamalar tarafından da kullanılan ortak kaynaklar ayırt edilir. Küme, çalışan düğümler, yük dengeleyicileri ve kalıcı diskler gibi kaynakların kaldırılma durumu sağlayıcı tarafında da kontrol edilir. Korunan kaynakların kullanım amacı ve olası maliyeti belirlenir; ortak kaynaklar üzerindeki diğer iş yüklerinin ihtiyaçları gözetilir.

Bulutta kaynak yönetimi, uygulamanın çalıştırılması kadar kullanımın sona ermesini de kapsar. Uygulamanın durdurulması, Kubernetes nesnelerinin silinmesi ve altyapı kaynaklarının kaldırılması farklı işlemlerdir. Gerekli verilerin korunması ve kullanılmayan kaynakların gerçekten kaldırıldığının doğrulanması, bu sürecin birlikte değerlendirilmesi gereken parçalarıdır.

7. Uygulama Çalışması

Bu çalışmada, Hafta 5'te kullanılan Ders Notları Kayıt Uygulaması Kubernetes üzerinde çalıştırılır. Öğrenci; uygulamanın dağıtımını, bileşenler arası iletişimi, veri kalıcılığını ve temel yönetim işlemlerini doğrular. Yerel çalışma tüm öğrenciler için zorunludur; bulut çalışması isteğe bağlıdır. Ortam hazırlığı Ek 1'de, yerel uygulama yönergeleri Ek 2'de ve bulut uygulaması yönergeleri Ek 3'te yer alır.

7.1. Yerel Uygulama

Yerel uygulama tüm öğrenciler için zorunludur. Hafta 5'te kullanılan Ders Notları Kayıt Uygulaması, bu hafta yerel Kubernetes kümesinde çalıştırılır. Web bileşeni ve PostgreSQL veritabanı korunur; önceki haftada Compose ile tanımlanan çalışma düzeni, Kubernetes nesneleriyle yeniden oluşturulur. Ortam hazırlığı Ek 1'de, dosyalar ve adım adım uygulama yönergeleri Ek 2'de yer alır. Öğrenci aşağıdaki görevleri gerçekleştirir:

1. **Uygulamayı Kubernetes üzerinde çalıştırma:** Web ve PostgreSQL bileşenlerini ayrı Pod'larda çalıştıracak Deployment tanımlarını hazırlar. Service nesneleriyle web uygulamasına erişimi ve bileşenler arasındaki iletişimi düzenler. Uygulama ayarlarını ConfigMap, gizli bilgileri Secret üzerinden sağlar; PostgreSQL için kalıcı depolama tanımlar.
2. **İşlevleri ve veri kalıcılığını doğrulama:** Tarayıcıdan kayıt ekler ve listeler. PostgreSQL Pod'u yeniden oluşturulduğunda, aynı kalıcı depolama kullanılarak önceki kayıtların korunduğunu gösterir.
3. **Sağlık ve yeniden oluşturma davranışlarını gözlemleme:** Sağlık kontrollerini ve kaynak gereksinimlerini tanımlar. Ek 2'deki kontrollü deneylerle hazır olma ve canlılık kontrollerinin farklı sonuçlarını gözlemler. Bir web Pod'u silindiğinde Kubernetes'in istenen kopya sayısını yeniden sağladığını doğrular.
4. **Web bileşenini ölçekleme:** Web kopya sayısını birden ikiye çıkarır, ardından yeniden bire indirir. Pod sayısındaki değişimi ve uygulamaya erişimin sürmesini gözlemler. PostgreSQL bu çalışmada tek kopya olarak kalır. Bu çalışmada kopya sayısı elle değiştirilir; otomatik ölçekleme kurulumu zorunlu değildir.
5. **Uygulamayı güncelleme ve önceki sürüme dönme:** Web bileşenini yeni imaj sürümüyle günceller; değişikliğin uygulamada görüldüğünü ve kayıtların korunduğunu doğrular. Ardından önceki sürüme dönerek kullanılan sürümü yeniden kontrol eder. Uygulama sürümüne dönüşün veritabanı kayıtlarını geri almadığını açıklar.

Başarı ölçütü yalnızca Pod'ların çalışıyor görünmesi değildir. Uygulamanın kullanılabilmesi, bileşenlerin iletişim kurması, verilerin korunması ve temel yönetim işlemlerinin beklenen sonuçları üretmesi birlikte gösterilir.

Doğrulama tamamlandıktan sonra uygulama kaynakları kontrollü biçimde kaldırılır. Öğrenci, kalıcı depolamanın korunması ile silinmesinin sonuçlarını ayırt eder; uygulama dosyalarını ve gözlem notlarını kendi bilgisayarında saklar.

7.2. Bulut Uygulaması

Bulut uygulaması isteğe bağlıdır. Bulut hesabı ve uygun kaynaklara erişimi olan öğrenciler, yerelde doğruladıkları uygulamayı AWS, Microsoft Azure veya Google Cloud üzerindeki bir

Kubernetes kümesinde çalıştırmaya teşvik edilir. Bir sağlayıcıda çalışma yapılması yeterlidir; yerel uygulamanın tamamlanması zorunlu kapsamı karşılar. Sağlayıcıya özgü hazırlık ve işlem yönergeleri Ek 3'te yer alır.

Öğrenci aşağıdaki görevleri gerçekleştirir:

1. **İmajı küme için erişilebilir hâle getirme:** Yerelde doğrulanan web imajını bir imaj kayıt deposuna gönderir. Bulut kümesinin imajı indirebilmesi için gerekli erişimi düzenler ve kullanılan sürümü belirtir.
2. **Uygulama tanımlarını buluta uyarlama:** Web ve PostgreSQL bileşenlerini hedef kümede çalıştırır. İmaj adresini, yapılandırmayı, gizli bilgileri ve kalıcı depolama ayarlarını bulut ortamına uygun biçimde düzenler.
3. **Erişimi ve uygulama işlevlerini doğrulama:** Web uygulamasına uzaktan erişir; kayıt ekleme ve listeleme işlemlerini gerçekleştirir. PostgreSQL'i doğrudan internete açmadan bileşenler arası iletişimi sağlar. Veritabanı Pod'u aynı kalıcı depolamayla yeniden oluşturulduğunda kayıtların korunduğunu kontrol eder.
4. **Çalışmayı tamamlayıp kaynakları kaldırma:** Doğrulama kanıtlarını aldıktan sonra saklanacak verileri belirler. Kullanılmayan uygulama ve bulut kaynaklarını kaldırır; küme, yük dengeleyicisi ve disk gibi ilişkili kaynakların durumunu ayrıca kontrol eder.

Bulutta yeni test kayıtları oluşturulması yeterlidir; yerel kayıtların aktarılması zorunlu değildir. Bununla birlikte öğrenci, imajın aktarılmasının veritabanı verilerini taşımadığını açıklayabilmelidir.

Yerelde gerçekleştirilen bütün deneylerin bulutta tekrarlanması gerekmez. Bu çalışmanın amacı, aynı uygulamanın farklı bir Kubernetes ortamında çalıştırılabildiğini göstermek ve ortama bağlı değişiklikleri ayırt etmektir.

7.3. Doğrulama, Teslim ve Değerlendirme

Öğrenci, yerel uygulamanın işlevlerini ve Kubernetes yönetim davranışlarını gözlemlenebilir kanıtlarla doğrular. Pod'ların çalışıyor görünmesi tek başına yeterli değildir. Uygulamaya erişim, kayıt işlemleri, veri kalıcılığı, ölçekleme ve sürüm değişiklikleri birlikte değerlendirilir. Doğrulama işlemlerinin ayrıntıları Ek 2'de yer alır.

Doğrulanması gereken sonuçlar

Yerel çalışmada aşağıdaki sonuçlar gösterilir:

- **Uygulamanın çalışması:** Web ve PostgreSQL bileşenleri kullanıma hazırdır; tarayıcıdan kayıt ekleme ve listeleme işlemleri gerçekleştirilir.

- **Veri kalıcılığı:** PostgreSQL Pod'u aynı kalıcı depolama kullanılarak yeniden oluşturulduğunda önceki kayıtlar görüntülenir. Yeniden oluşturma öncesinde ve sonrasında aynı kayıtların bulunduğu gösterilir.
- **Sağlık ve yeniden oluşturma davranışı:** Hazır olma ve canlılık kontrollerinin farklı sonuçları, Ek 2'deki kontrollü deneylerle gösterilir. Silinen web Pod'unun yerine yeni bir Pod oluşturulduğu gözlemlenir.
- **Ölçekleme:** Web kopya sayısı birden ikiye, ardından yeniden bire değiştirilir. Beklenen Pod sayısına ulaşıldığı ve uygulamanın erişilebilir olduğu gösterilir.
- **Güncelleme ve geri dönüş:** Yeni web imajı sürümündeki değişiklik uygulamada görülür; önceki sürüme dönüş doğrulanır. Bu işlemler sırasında mevcut kayıtların korunduğu gösterilir.
- **Çalışmanın sonlandırılması:** Uygulama kaynaklarının kaldırıldığı, kalıcı depolamanın korunduğu veya silindiği belirtilir.

Teslim içeriği

Öğrenci, aşağıdaki iki bileşenden oluşan kısa bir uygulama raporu teslim eder:

1. **Doğrulama kanıtları:** Yukarıdaki sonuçları gösteren ekran görüntüleri veya ilgili araç çıktıları sunulur. Her kanıtın altında, yapılan değişikliği ve gözlenen sonucu açıklayan kısa bir metin bulunur. Aynı sonucu tekrar eden çok sayıda görüntü yerine, sonucu açıkça gösteren kanıtlar seçilir.
2. **Kavramsal açıklamalar:** Öğrenci aşağıdaki soruları kendi uygulamasındaki nesneler ve gözlemler üzerinden yanıtlar:
    - Web uygulamasına erişim ile web bileşeninin PostgreSQL'e erişimi nasıl sağlanır?
    - Silinen web Pod'unun yerine neden yeni bir Pod oluşturulur? Konteynerin yeniden başlatılmasıyla Pod'un yeniden oluşturulması arasında ne fark vardır?
    - Kayıtlar nerede saklanır? PostgreSQL Pod'u yeniden oluşturulduğunda neden korunur?
    - Hazır olma kontrolünün başarısızlığı ile canlılık kontrolünün başarısızlığı uygulamayı nasıl etkiler?
    - Web kopya sayısı değiştiğinde hangi bileşenler değişir? PostgreSQL neden aynı şekilde çoğaltılmaz?

- Yeni sürüme geçiş ve önceki sürüme dönüş nasıl doğrulanır? Önceki uygulama sürümüne dönmek neden veritabanını da önceki hâline getirmez?

Hazır kaynak kodların ve Kubernetes yapılandırma dosyalarının ayrıca teslim edilmesi zorunlu değildir. Sorunların incelenmesi gerektiğinde ilgili dosyalar istenebilir. Öğrenci çalışma dosyalarını kendi bilgisayarında korur. Parolalar, erişim anahtarları ve diğer hassas bilgiler raporda veya ekran görüntülerinde bulunmamalıdır.

Değerlendirme ölçütleri

Değerlendirmede, uygulamanın doğru çalışması ile öğrencinin gözlenen davranışları açıklayabilmesi birlikte dikkate alınır. Bileşenler arası iletişimin sağlanması, yapılandırmanın uygun kullanılması, verinin korunması ve temel yönetim işlemlerinin doğrulanması esas alınır.

Beklenen sonucu vermeyen bir deneyde öğrenci, gördüğü belirtiyi, incelediği durum bilgilerini ve yaptığı düzeltmeyi açıklar. Hatanın nedenini doğru belirlemek ve düzeltme sonrasında sonucu yeniden doğrulamak da değerlendirmeye katkı sağlar.

Bulut çalışmasını gerçekleştiren öğrenciler rapora seçtikleri sağlayıcıyı, kullanılan imaj sürümünü, uygulamaya erişim ve kayıt işlemlerine ilişkin kanıtları ve kaynakların kaldırılma durumunu ekler. Bulut çalışması yapmamak, zorunlu yerel çalışma için değerlendirme kaybına neden olmaz.

8. Bölüm Özeti

Bu hafta, konteyner uygulamalarının Kubernetes üzerinde dağıtılması ve yönetilmesi ele alındı. Kubernetes'in bildirimsel yönetim yaklaşımıyla uygulamanın istenen durumunu korumaya çalıştığı; dağıtım, iletişim, yapılandırma, depolama, sağlık, ölçekleme ve güncelleme gereksinimlerinin farklı nesne ve mekanizmalarla karşılandığı açıklandı.

Kubernetes'in temelinde bildirimsel yönetim yaklaşımı bulunur. Kullanıcı, uygulamanın istenen durumunu nesne tanımlarıyla belirtir; Kubernetes mevcut durumu bu tanımlara uygun hâle getirmeye çalışır. Kontrol düzlemi kümenin yönetimini yürütürken çalışan düğümler uygulama Pod'larını çalıştırır. Ancak istenen durumun gerçekleşmesi, yeterli kaynakların ve doğru yapılandırmanın bulunmasına bağlıdır.

Pod, uygulama konteynerlerinin çalıştığı temel birimdir. Deployment, Pod'ların oluşturulmasını, kopya sayısının korunmasını ve güncellenmesini yönetir. Service ise değişebilen Pod adreslerinden bağımsız bir erişim noktası sağlar. Kullanıcının web uygulamasına erişimi ile web bileşeninin veritabanına erişimi ayrı bağlantılardır ve ayrı değerlendirilir.

Uygulama imajı, yapılandırma ve kalıcı veri farklı görevler üstlenir. ConfigMap, gizli olmayan ayarları; Secret, parola ve uygulama sırları gibi bilgileri uygulamaya sağlar. Secret kullanılması tek başına bütün güvenlik gereksinimlerini karşılamaz. Kalıcı depolama ise gerekli verilerin Pod'un yaşam döngüsünden bağımsız saklanmasını sağlar. Verinin kalıcı olması, yedeklendiği veya her türlü arızaya karşı korunduğu anlamına gelmez.

Sağlık kontrollerinde başlangıç, canlılık ve hazır olma durumları birbirinden ayrılır. Canlılık kontrolü gerektiğinde konteynerin yeniden başlatılmasını sağlarken hazır olma kontrolü Pod'un Service üzerinden trafik alıp almamasını etkiler. Bir Pod'un çalışması, uygulamanın kullanıma hazır veya bütün işlevlerinin doğru olduğu anlamına gelmez. Bu nedenle sağlık bilgileri, uygulama üzerinden yapılan işlevsel kontrollerle birlikte değerlendirilir.

Kaynak istekleri ve sınırlar, uygulamanın kaynak gereksinimlerini ve kullanım sınırlarını tanımlar. Web bileşeninin kopya sayısı artırılarak yatay ölçekleme yapılabilir; ancak bu işlem veritabanını kendiliğinden ölçeklemez veya performansın aynı oranda artmasını garanti etmez. Yeni imaj sürümüne geçiş ve önceki sürüme dönüş Deployment üzerinden yönetilir. Önceki uygulama sürümüne dönmek, veritabanı kayıtlarını veya şema değişikliklerini otomatik olarak geri almaz.

Yerel ortamda öğrenilen Kubernetes kavramları bulutta da geçerlidir. Bununla birlikte imaj erişimi, yapılandırma, ağ ve depolama ayarları hedef ortama göre uyarlanır. Yönetilen Kubernetes hizmetleri altyapı yönetiminin bir bölümünü sağlayıcıya bırakır; uygulamanın doğru yapılandırılması ve doğrulanması kullanıcı sorumluluğunda kalır. İmajın buluta aktarılması, yerel verilerin de aktarılması anlamına gelmez.

Kubernetes üzerinde bir uygulamayı yönetmek, yalnızca Pod'ların çalışmasını sağlamakla sınırlı değildir. Uygulamaya erişimin, bileşenler arası iletişimin, veri kalıcılığının ve sürüm değişikliklerinin beklenen sonuçları verdiği de doğrulanmalıdır. Uygulama nesneleri, kalıcı veriler ve küme altyapısı farklı yaşam döngülerine sahip olduğundan, kullanım sonunda bu kaynaklar ayrı ayrı değerlendirilir.

Kaynakça

[1] "Overview," Kubernetes. Accessed: Oct. 07, 2026. [Online]. Available: https://kubernetes.io/docs/concepts/overview/
[2] "Objects In Kubernetes," Kubernetes. Accessed: Oct. 07, 2026. [Online]. Available: https://kubernetes.io/docs/concepts/overview/working-with-objects/
[3] "Controllers," Kubernetes. Accessed: Oct. 07, 2026. [Online]. Available: https://kubernetes.io/docs/concepts/architecture/controller/
[4] "Kubernetes Components," Kubernetes. Accessed: Oct. 07, 2026. [Online]. Available: https://kubernetes.io/docs/concepts/overview/components/
[5] "Pods," Kubernetes. Accessed: Oct. 07, 2026. [Online]. Available: https://kubernetes.io/docs/concepts/workloads/pods/

[6] “Deployments,” Kubernetes. Accessed: Oct. 07, 2026. [Online]. Available: https://kubernetes.io/docs/concepts/workloads/controllers/deployment/
[7] “ReplicaSet,” Kubernetes. Accessed: Oct. 07, 2026. [Online]. Available: https://kubernetes.io/docs/concepts/workloads/controllers/replicaset/
[8] “Namespaces,” Kubernetes. Accessed: Oct. 07, 2026. [Online]. Available: https://kubernetes.io/docs/concepts/overview/working-with-objects/namespaces/
[9] “Labels and Selectors,” Kubernetes. Accessed: Oct. 07, 2026. [Online]. Available: https://kubernetes.io/docs/concepts/overview/working-with-objects/labels/
[10] “Explore the Kubernetes view,” Docker Documentation. Accessed: Oct. 08, 2026. [Online]. Available: https://docs.docker.com/desktop/use-desktop/kubernetes/
[11] “minikube start,” minikube. Accessed: Oct. 08, 2026. [Online]. Available: https://minikube.sigs.k8s.io/docs/start/
[12] “Connect kubectl to an EKS cluster by creating a kubeconfig file - Amazon EKS.” Accessed: Oct. 08, 2026. [Online]. Available: https://docs.aws.amazon.com/eks/latest/userguide/create-kubeconfig.html
[13] davidsmatlak, “Deploy an Azure Kubernetes Service (AKS) Cluster using Azure CLI - Azure Kubernetes Service.” Accessed: Oct. 08, 2026. [Online]. Available: https://learn.microsoft.com/en-us/azure/aks/learn/quick-kubernetes-deploy-cli
[14] “Install kubectl and configure cluster access | Google Kubernetes Engine (GKE),” Google Cloud Documentation. Accessed: Oct. 08, 2026. [Online]. Available: https://docs.cloud.google.com/kubernetes-engine/docs/how-to/cluster-access-for-kubectl
[15] “Command line tool (kubectl),” Kubernetes. Accessed: Oct. 08, 2026. [Online]. Available: https://kubernetes.io/docs/reference/kubectl/
[16] “Organizing Cluster Access Using kubeconfig Files,” Kubernetes. Accessed: Oct. 08, 2026. [Online]. Available: https://kubernetes.io/docs/concepts/configuration/organize-cluster-access-kubeconfig/
[17] “kubectl Quick Reference,” Kubernetes. Accessed: Oct. 08, 2026. [Online]. Available: https://kubernetes.io/docs/reference/kubectl/quick-reference/
[18] “Images,” Kubernetes. Accessed: Oct. 08, 2026. [Online]. Available: https://kubernetes.io/docs/concepts/containers/images/
[19] “Service,” Kubernetes. Accessed: Oct. 08, 2026. [Online]. Available: https://kubernetes.io/docs/concepts/services-networking/service/
[20] “DNS for Services and Pods,” Kubernetes. Accessed: Oct. 08, 2026. [Online]. Available: https://kubernetes.io/docs/concepts/services-networking/dns-pod-service/
[21] “Connecting Applications with Services,” Kubernetes. Accessed: Oct. 08, 2026. [Online]. Available: https://kubernetes.io/docs/tutorials/services/connect-applications-service/
[22] “Use Port Forwarding to Access Applications in a Cluster,” Kubernetes. Accessed: Oct. 08, 2026. [Online]. Available: https://kubernetes.io/docs/tasks/access-application-cluster/port-forward-access-application-cluster/
[23] “ConfigMaps,” Kubernetes. Accessed: Oct. 08, 2026. [Online]. Available: https://kubernetes.io/docs/concepts/configuration/configmap/
[24] “Secrets,” Kubernetes. Accessed: Oct. 08, 2026. [Online]. Available: https://kubernetes.io/docs/concepts/configuration/secret/

[25] "Volumes," Kubernetes. Accessed: Oct. 08, 2026. [Online]. Available: https://kubernetes.io/docs/concepts/storage/volumes/
[26] "Persistent Volumes," Kubernetes. Accessed: Oct. 08, 2026. [Online]. Available: https://kubernetes.io/docs/concepts/storage/persistent-volumes/
[27] "Storage Classes," Kubernetes. Accessed: Oct. 08, 2026. [Online]. Available: https://kubernetes.io/docs/concepts/storage/storage-classes/
[28] "Liveness, Readiness, and Startup Probes," Kubernetes. Accessed: Oct. 08, 2026. [Online]. Available: https://kubernetes.io/docs/concepts/workloads/pods/probes/
[29] "Configure Liveness, Readiness and Startup Probes," Kubernetes. Accessed: Oct. 08, 2026. [Online]. Available: https://kubernetes.io/docs/tasks/configure-pod-container/configure-liveness-readiness-startup-probes/
[30] "Pod Lifecycle," Kubernetes. Accessed: Oct. 08, 2026. [Online]. Available: https://kubernetes.io/docs/concepts/workloads/pods/pod-lifecycle/
[31] "Resource Management for Pods and Containers," Kubernetes. Accessed: Oct. 08, 2026. [Online]. Available: https://kubernetes.io/docs/concepts/configuration/manage-resources-containers/
[32] "Assign CPU Resources to Containers and Pods," Kubernetes. Accessed: Oct. 08, 2026. [Online]. Available: https://kubernetes.io/docs/tasks/configure-pod-container/assign-cpu-resource/
[33] "Assign Memory Resources to Containers and Pods," Kubernetes. Accessed: Oct. 08, 2026. [Online]. Available: https://kubernetes.io/docs/tasks/configure-pod-container/assign-memory-resource/
[34] "Horizontal Pod Autoscaling," Kubernetes. Accessed: Oct. 08, 2026. [Online]. Available: https://kubernetes.io/docs/concepts/workloads/autoscaling/horizontal-pod-autoscale/
[35] "Performing a Rolling Update," Kubernetes. Accessed: Oct. 09, 2026. [Online]. Available: https://kubernetes.io/docs/tutorials/kubernetes-basics/update/update-intro/
[36] "What is Amazon EKS? - Amazon EKS." Accessed: Oct. 09, 2026. [Online]. Available: https://docs.aws.amazon.com/eks/latest/userguide/what-is-eks.html
[37] davidsmatlak, "What is Azure Kubernetes Service (AKS)? - Azure Kubernetes Service." Accessed: Oct. 09, 2026. [Online]. Available: https://learn.microsoft.com/en-us/azure/aks/what-is-aks
[38] "GKE overview | Google Kubernetes Engine (GKE)," Google Cloud Documentation. Accessed: Oct. 09, 2026. [Online]. Available: https://docs.cloud.google.com/kubernetes-engine/docs/concepts/kubernetes-engine-overview
[39] "Delete a cluster - Amazon EKS." Accessed: Oct. 09, 2026. [Online]. Available: https://docs.aws.amazon.com/eks/latest/userguide/delete-cluster.html
[40] davidsmatlak, "Delete an Azure Kubernetes Service (AKS) cluster - Azure Kubernetes Service." Accessed: Oct. 09, 2026. [Online]. Available: https://learn.microsoft.com/en-us/azure/aks/delete-cluster
[41] "Deleting a cluster | Google Kubernetes Engine (GKE)," Google Cloud Documentation. Accessed: Oct. 09, 2026. [Online]. Available: https://docs.cloud.google.com/kubernetes-engine/docs/how-to/deleting-a-cluster

Hafta 6 – Çalışma Soruları

Soruları ders notuna bakmadan yanıtlamaya çalışınız. Yanıtlarınızda yalnızca kavramların tanımlarını değil, ilgili mekanizmanın neden ve nasıl kullanıldığını da açıklayınız. Komut veya yapılandırma dosyası yazmanız gerekli değildir.

A. Kubernetes ve Bildirimsel Yönetim

1. Bir uygulamayı konteyner içinde çalıştırmak ile Kubernetes üzerinden yönetmek arasındaki fark nedir? Birden fazla çalışma örneğinin yönetilmesi, hata sonrasında yeniden oluşturma ve sürüm güncelleme açısından Kubernetes'in rolünü açıklayınız.
2. Bildirimsel yönetim yaklaşımında kullanıcı hangi bilgileri tanımlar, Kubernetes hangi görevi üstlenir? Üç web kopyası istendiği hâlde yalnızca iki kopya çalışıyorsa Kubernetes'in beklenen davranışını açıklayınız. Üçüncü kopyanın hemen çalışamamasına yol açabilecek iki neden belirtiniz.
3. Kontrol düzlemi ile çalışan düğümlerin görevlerini karşılaştırınız. Bir Pod'un hangi düğümde çalışacağına karar verilmesi ile konteynerin o düğümde çalıştırılması hangi bileşenlerin sorumluluğundadır?

B. Pod, Deployment ve Service İlişkisi

4. Pod, ReplicaSet ve Deployment arasındaki ilişkiyi açıklayınız. Deployment tarafından yönetilen bir web Pod'u silindiğinde neden yeni bir Pod oluşturulur? Tek başına oluşturulan bir Pod için aynı davranış beklenir mi?
5. İki web Pod'unun adresleri, Pod'lar yeniden oluşturulduğunda değişmektedir. Kullanıcıların veya diğer uygulama bileşenlerinin doğrudan bu adreslere bağlanması neden sorun oluşturabilir? Service, etiketler ve seçiciler bu sorunun çözümünde nasıl birlikte kullanılır?
6. Bir randevu uygulamasında web bileşeni ile veritabanı ayrı Pod'larda çalışmaktadır. Kullanıcının web uygulamasına erişimi ile web bileşeninin veritabanına erişimi nasıl ayrılmalıdır? Veritabanı bağlantısında neden web Pod'unun localhost adresi kullanılmamalıdır? Veritabanını internete açmak gerekli midir?

C. Yapılandırma ve Kalıcı Veri

7. Bir uygulamanın çalışma ortamı adı, veritabanı adresi, veritabanı parolası ve uygulama sırrı bulunmaktadır. Bu bilgileri ConfigMap ve Secret arasında nasıl ayırırsınız? Bilgilerin imajdan ayrı tutulması hangi kolaylıkları sağlar? Secret kullanılması neden tek başına tam güvenlik anlamına gelmez?
8. Bir veritabanı Pod'u yeniden oluşturulduktan sonra uygulamada eski kayıtlar görünmemektedir. Bu durum verilerin kesinlikle silindiğini gösterir mi? Kalıcı

depolama bağlantısı, kullanılan veri dizini ve farklı bir depolama alanına bağlanma olasılığı üzerinden inceleme yaklaşımınızı açıklayınız.

9. PersistentVolume, PersistentVolumeClaim ve StorageClass arasındaki ilişkiyi açıklayınız. Bir depolama isteğinin oluşturulması neden her durumda hemen kullanılabilir bir depolama alanı elde edildiği anlamına gelmez? Ayrıca kalıcı depolama kullanılması ile yedekleme arasındaki farkı belirtiniz.

D. Sağlık Kontrolleri ve Kaynak Yönetimi

10. Başlangıç, canlılık ve hazır olma kontrollerinin amaçlarını karşılaştırınız. Başlaması uzun süren bir uygulamada başlangıç kontrolü neden yararlıdır? Hazır olma kontrolünün başarısızlığı ile canlılık kontrolünün başarısızlığı hangi farklı sonuçlara yol açar?

11. Bir web Pod'u çalışıyor görünmesine rağmen Service üzerinden kullanıcı isteği alamamaktadır. Bunun olası nedenleri nelerdir? Hazır olma durumu, Service seçicileri ve port bilgileri açısından hangi kontrolleri yaparsınız? Yalnızca Pod'un çalışıyor olması neden yeterli kanıt değildir?

12. Kaynak isteği ile kaynak sınırı arasındaki farkı açıklayınız. Bir Pod'un kaynak isteğini karşılayabilecek düğüm bulunmaması ile çalışan bir konteynerin bellek sınırını aşması neden farklı durumlardır? İşlemci sınırına ulaşılması ile bellek sınırının aşılması da aynı sonucu mu doğurur?

E. Ölçekleme, Güncelleme ve Geri Dönüş

13. Bir duyuru uygulamasının web kopya sayısı birden dörde çıkarılmış, ancak yanıt sürelerinde beklenen iyileşme görülmemiştir. Web bileşeninin ölçeklenmesi neden performansı aynı oranda artırmayabilir? Ortak veritabanı ve küme kapasitesi açısından açıklayınız. Kopya sayısının elle değiştirilmesi ile otomatik ölçekleme arasındaki farkı da belirtiniz.

14. Bir web uygulamasının yeni imaj sürümü Deployment üzerinden dağıtılacaktır. Aşamalı güncellemenin temel yaklaşımını açıklayınız. Yeni Pod'lar kullanıma hazır olamazsa güncellemenin ilerlemesi nasıl etkilenebilir? Aşamalı güncelleme neden her koşulda kesintisiz hizmet garantisi değildir?

15. Yeni web sürümü sorun oluşturduğu için önceki Deployment sürümüne dönülmüştür. Ancak yeni sürümün veritabanında yaptığı bir değişiklik hâlâ bulunmaktadır. Bu durum neden mümkündür? Geri dönüşün Pod şablonu, imaj sürümü ve kalıcı veriler üzerindeki kapsamını açıklayınız.

F. Buluta Geçiş ve Bütünleşik Değerlendirme

16. Yerel Kubernetes kümesinde çalışan bir uygulama, buluttaki bir Kubernetes kümesine taşınacaktır. İmaj erişimi, yapılandırma, kalıcı depolama ve dış erişim

bakımından hangi bilgiler yeniden değerlendirilmelidir? Yerel bilgisayarda bulunan imajın ve veritabanı kayıtlarının bulut kümesinde kendiliğinden bulunacağı varsayılabilir mi?

17. Yönetilen Kubernetes hizmetinde kontrol düzleminin sağlayıcı tarafından işletilmesi, uygulamanın bütün yönetim sorumluluklarının da sağlayıcıya geçtiği anlamına gelir mi? Uygulama yapılandırması, sağlık kontrolleri, kaynak gereksinimleri ve veri korunması açısından açıklayınız. Deneme sonunda uygulamanın Deployment'ını silmek neden bütün bulut maliyetlerini sonlandırmayabilir?

18. Bir üniversitenin laboratuvar rezervasyon sistemi, web bileşeni ve PostgreSQL veritabanından oluşmaktadır. Sistem önce yerel Kubernetes kümesinde çalıştırılacak, ardından isteğe bağlı olarak bulutta denenecektir. Aşağıdaki gereksinimleri karşılayan bir tasarımı açıklayınız:

    - Kullanıcılar web uygulamasına erişebilmeli; veritabanı doğrudan internete açılmamalıdır.
    - Web Pod'ları yeniden oluşturulduğunda bileşenler arasındaki iletişim sürdürülebilmelidir.
    - Veritabanı Pod'u yeniden oluşturulduğunda rezervasyon kayıtları korunmalıdır.
    - Parolalar imajın içine eklenmemelidir.
    - Kullanıma hazır olmayan web Pod'larına yeni kullanıcı trafiği gönderilmemelidir.
    - Web kopya sayısı artırılabilmeli; yeni sürüm dağıtılıp gerektiğinde önceki sürüme dönülebilmelidir.
    - Bulut denemesi tamamlandığında gerekli veriler korunmalı ve kullanılmayan kaynaklar kaldırılmalıdır.

Yanıtınızda gerekli Kubernetes nesnelerini ve aralarındaki ilişkileri belirtiniz. Tasarımın doğru çalıştığını göstermek için yapacağınız en az dört doğrulama kontrolünü, beklenen sonuçlarıyla birlikte açıklayınız.
